Team Control Number
IMMC26563020

2026
IMMC
Summary Sheet

Topology and Fractal Optimization of Heat Transfer Fins

Summary

Incorporating fins into heat storage tanks represents an effective approach to overcome the inherently low thermal conductivity of phase change materials (PCMs) and significantly enhance heat transfer efficiency. This paper presents comprehensive numerical methodologies employing topology and fractal optimization techniques to determine optimal fin configurations and spatial distributions.

For **Task 1**, we establish a **two-dimensional finite element model (FEM)** based on computational fluid dynamics (CFD) to investigate the heat transfer process within PCM-filled storage tanks. The enthalpy-porosity method is employed to characterize the phase change behavior of the PCM. Our simulations reveal that phase change initiates around the rectangular fins and subsequently propagates toward the outer tank wall, transforming the PCM from solid to liquid phase. Considering PCM temperature elevation from 293K to 333K, the initial phase change times (t_1) at interval angles (θ) of 72°, 60°, 45°, 36°, 30°, and 24° are 47.6min, 39.2min, 31.4min, 25.2min, 20.7min, and 17.2min respectively, while complete phase change times (t_2) are 158.5min, 142.9min, 123.4min, 106.2min, 92.4min, and 81.9min respectively. The initial phase change time (t_1), liquid fraction (f_m), and enhancement ratio (ER) serve as evaluation indicators for thermal performance assessment. The optimal interval angle is determined to be 24° with minimum t_1 of 17.2min and maximum ER of 12.3%.

For **Task 2**, we systematically investigate the effects of interval angle (θ), fin length (L), and fin width (W) on triangular fin performance. Results demonstrate that $\theta = 24$ achieves 63.83%, 56.34%, 46.73%, 46.73%, and 31.71% heating rate improvements compared with $\theta = 72$, 60°, 45°, 36°, and 30° respectively. Furthermore, $L = 0.024$m enhances heating rates by 42.17%, 70.04%, and 79.52% compared with $L = 0.006$m, 0.012m, and 0.018m respectively. Additionally, $W = 0.01$m increases heating rates by 7.60%, 22.55%, and 35.91% compared with $W = 0.008$m, 0.006m, and 0.004m respectively. The priority order of fin parameters is established as: $L > \theta > W$. Rectangular fins demonstrate superior heating efficiency, achieving 11.95%, 14.99%, 15.43%, and 15.62% improvements over triangular fins at $\theta = 72$, 45°, 30°, and 24° respectively.

For **Task 3**, two distinct optimization methodologies are proposed to achieve optimal fin distribution: **topology optimization** and **fractal optimization**. The topology optimization approach employs the variable density method with subsequent image reconstruction to maximize average temperature. The fractal optimization generates optimal parent-children fractal tree structures based on Murray's law and growth ratio principles. Results indicate that topology optimization enhances heating rates by 18.84% and 28.17% compared with rectangular and triangular fins respectively, while fractal optimization achieves improvements of 14.01% and 24.25%.

For **Sensitivity Analysis**, the robustness of the fractal optimization model is validated by varying the exponent γ in Murray's law across values of 2, 3, and 4. Results demonstrate that temperature evolution exhibits minimal variation under different γ values, attributable to the inherent spacing control mechanism of the fractal tree structure. This confirms that the fractal design model possesses good stability and is insensitive to the bifurcation parameter, ensuring reliable performance across a range of design configurations.

Keywords: Heat Transfer Fins; CFD; Topology Optimization; Fractal Geometry; Phase Change Materials; Enthalpy-Porosity Method; ANSYS Fluent

Contents

1 Introduction **3**
1.1 Problem Background . . . 3
1.2 Our Work . . . 3

2 Assumptions and Justifications **3**

3 Notations **4**

4 Task 1: Phase Change Model and Heat Transfer Analysis **5**
4.1 Phase Change Model Development . . . 5
4.1.1 Governing Equations . . . 5
4.1.2 Initial and Boundary Conditions . . . 7
4.1.3 Numerical Procedure . . . 7
4.2 Heat Transfer Process Analysis . . . 8
4.2.1 Temperature Distribution . . . 8
4.2.2 Phase Change Time . . . 9
4.2.3 Liquid Fraction Analysis . . . 10
4.3 Optimization of the Interval Angle . . . 11
4.3.1 Enhancement Ratio . . . 11
4.3.2 Phase Change Time Analysis . . . 11

5 Task 2: Parametric Study of Triangular Fins **12**
5.1 Effect of the Interval Angle . . . 12
5.1.1 Influence on Heat Transfer . . . 12
5.1.2 Comparison with Rectangular Fins . . . 13
5.2 Effect of the Fin Length . . . 13
5.2.1 Influence on Heat Transfer . . . 14
5.2.2 Comparison and Analysis . . . 14
5.3 Effect of the Fin Width . . . 14
5.3.1 Influence on Heat Transfer . . . 15
5.3.2 Comparison and Analysis . . . 15

6 Task 3: Optimal Fin Distribution via Topology and Fractal Optimization **16**
6.1 Topology Optimization . . . 16
6.1.1 Variable Density Method . . . 16
6.1.2 Optimization Result . . . 17
6.2 Fractal Optimization . . . 19
6.2.1 Murray's Law and Branch Generation . . . 19
6.2.2 Optimization Result . . . 20
6.2.3 Heat Transfer Process . . . 20
6.3 Comparison of Two Optimization Models . . . 20

7 Sensitivity Analysis **21**

8 Strengths and Weaknesses **21**
8.1 Strengths . . . 21
8.2 Weaknesses . . . 21

References **22**

Appendices **22**

1 Introduction

1.1 Problem Background

A double-wall tank incorporating phase change materials (PCMs) represents a widely adopted configuration for thermal energy storage applications. In such systems, heated transfer fluid enters the tank and releases thermal energy to the PCM, which absorbs heat through phase transformation from solid to liquid state and subsequently releases heat during the reverse solidification process. To accelerate heat transfer rates within these systems, fins are strategically embedded within the PCM region between tank walls.

The incorporation of fins significantly reduces melting time and substantially increases energy storage rates. The low thermal conductivity inherent to most PCMs constitutes the primary limitation affecting heat transfer efficiency. Consequently, determining the optimal fin distribution represents a critical engineering challenge for enhancing heat transfer performance in thermal storage systems.

This research addresses the fundamental need to develop mathematical models and optimization methodologies capable of identifying optimal fin configurations that maximize heat transfer efficiency while considering practical manufacturing constraints.

1.2 Our Work

Our systematic approach to addressing the comprehensive heat transfer fin optimization challenge encompasses the following methodology:

Task 1: We establish a two-dimensional finite element model based on computational fluid dynamics to simulate the heat transfer process within the storage tank. The enthalpy-porosity method is employed to describe PCM phase change behavior. Comprehensive parametric studies are conducted to investigate the effect of fin interval angle on heat transfer characteristics, and transfer times are calculated for PCM temperature changes from 293K to 333K.

Task 2: Building upon the analysis framework established in Task 1, we extend our investigation to triangular fin configurations. The effects of fin geometry parameters including interval angle, length, and width on heat transfer performance are systematically examined. Comparative analysis between rectangular and triangular fin configurations is performed to quantify relative performance differences.

Task 3: We develop and implement two distinct mathematical optimization models to determine optimal fin spatial distributions. Topology optimization employing the variable density method is applied to generate optimized irregular fin structures. Additionally, fractal optimization based on Murray's law is implemented to produce regular branching fin configurations suitable for practical manufacturing.

Task 4: Based on comprehensive simulation results and structural analysis, we formulate a recommendation letter for fin manufacturing companies, providing practical design guidelines and parameter ranges for optimal thermal performance.

2 Assumptions and Justifications

To ensure model tractability while maintaining practical applicability to real thermal storage systems, we establish the following assumptions:

Assumption **1: The physical properties of the PCM are homogeneous, isotropic, and temperature independent (except for density in the liquid state).**
Justification: This assumption enables the application of standard governing equations and simplifies the numerical solution procedure while maintaining reasonable accuracy for engineering applications.

Assumption **2: The phase change process of PCM is regarded as laminar, unsteady, and incompressible.**

Justification: The relatively low velocities encountered during natural convection in PCM systems justify the laminar flow assumption, while incompressibility is appropriate for liquid-phase behavior.

***Assumption* 3: Natural convection of liquid PCM conforms to the Boussinesq assumption.**
Justification: The Boussinesq approximation provides accurate results when density variations are small relative to the reference density, which is valid for most PCM applications within typical operating temperature ranges.

***Assumption* 4: The liquid fraction is assumed to change linearly with temperature within the mushy zone.**
Justification: This linearization simplifies the computational treatment of the phase change interface while providing reasonable approximations for most PCM materials.

***Assumption* 5: The effect of viscous dissipation is neglected during the simulation.**
Justification: Viscous dissipation effects are typically negligible in natural convection-dominated flows with moderate velocities characteristic of PCM melting processes.

***Assumption* 6: Heat loss to the environment is neglected.**
Justification: This assumption isolates the internal heat transfer mechanisms for systematic study and represents well-insulated storage tank conditions.

3 Notations

Table 1 summarizes the key mathematical symbols employed throughout this paper.

Table 1: Principal notations and their descriptions

Symbol	Description	Unit
L	Fin length	m
W	Fin width	m
θ	Interval angle	°
T	Temperature	K
t_1	Initial phase change time	min
t_2	Complete phase change time	min
R_1	Radius of the inner wall	m
R_2	Radius of the outer wall	m
ρ	Density	kg/m^3
f_m	Liquid fraction	-
λ	Thermal conductivity	W/(m·K)
E_R	Enhancement ratio	-
η	Penalty factor	-
α	Fractal length ratio	-
c	Specific heat	J/(kg·K)
μ	Dynamic viscosity	kg/(m·s)
δ	Thermal expansion coefficient	K^{-1}
p	Pressure	Pa
u, v	Velocity components	m/s

4 Task 1: Phase Change Model and Heat Transfer Analysis

4.1 Phase Change Model Development

A two-dimensional finite element model (FEM) was established to investigate the thermal and transient responses during the heat transfer process. The phase change behavior of the PCM was characterized using the enthalpy-porosity method. ANSYS Fluent, based on Computational Fluid Dynamics (CFD) methodology, was employed to simulate the heat transfer process.

4.1.1 Governing Equations

The heat storage tank configuration and representative cross-section are illustrated in Figure 1. The interior domain Ω_1 represents the heat source maintained at 373K, while the exterior domain Ω_3 is adiabatic. The intermediate domain Ω_2 contains the phase change material.

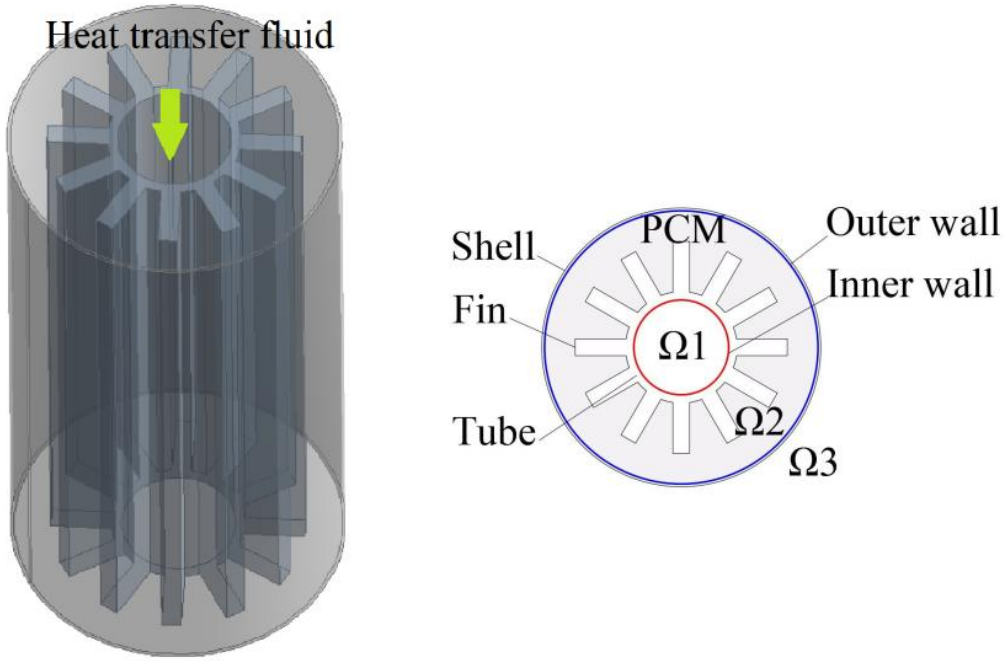

Figure 1: Schematic diagram of heat storage tank and cross-section configuration

The enthalpy-porosity method is applied to describe the phase change process within the heat storage tank[1]. The governing equations for continuity, momentum, and energy predict flow behavior, heat transfer characteristics, and PCM melting progression respectively[2].

1) Continuity equation:

$$\frac{\partial u}{\partial x}+\frac{\partial v}{\partial y}=0 \tag{1}$$

2) Momentum equations:

$$\frac{\partial(\rho_{\mathrm{PCM}}u)}{\partial t}+u\frac{\partial(\rho_{\mathrm{PCM}}u)}{\partial x}+v\frac{\partial(\rho_{\mathrm{PCM}}u)}{\partial y}=\frac{\partial}{\partial x}\left(\mu_{\mathrm{PCM}}\frac{\partial u}{\partial x}\right)+\frac{\partial}{\partial y}\left(\mu_{\mathrm{PCM}}\frac{\partial u}{\partial y}\right)-\frac{\partial p}{\partial x}+S_u \tag{2}$$

$$\frac{\partial(\rho_{\mathrm{PCM}}v)}{\partial t}+u\frac{\partial(\rho_{\mathrm{PCM}}v)}{\partial x}+v\frac{\partial(\rho_{\mathrm{PCM}}v)}{\partial y}=\frac{\partial}{\partial x}\left(\mu_{\mathrm{PCM}}\frac{\partial v}{\partial x}\right)+\frac{\partial}{\partial y}\left(\mu_{\mathrm{PCM}}\frac{\partial v}{\partial y}\right)-\frac{\partial p}{\partial y}+S_v \tag{3}$$

3) Energy equations:

For PCM:

$$\rho_{\mathrm{PCM}} c_{\mathrm{PCM}} \frac{\partial T}{\partial t} + \rho_{\mathrm{PCM}} c_{\mathrm{PCM}} \left(u \frac{\partial T}{\partial x} + v \frac{\partial T}{\partial y} \right) = \frac{\partial}{\partial x} \left(\lambda_{\mathrm{PCM}} \frac{\partial T}{\partial x} \right) + \frac{\partial}{\partial y} \left(\lambda_{\mathrm{PCM}} \frac{\partial T}{\partial y} \right) + S_e \tag{4}$$

For fin:

$$\rho_{\mathrm{fin}} c_{\mathrm{fin}} \frac{\partial T}{\partial t} = \frac{\partial}{\partial x} \left(\lambda_{\mathrm{fin}} \frac{\partial T}{\partial x} \right) + \frac{\partial}{\partial y} \left(\lambda_{\mathrm{fin}} \frac{\partial T}{\partial y} \right) \tag{5}$$

where T denotes temperature, p represents pressure, u and v are velocity components in the x and y directions respectively, c is specific heat, λ is thermal conductivity, and ρ is density. The source terms S_u, S_v, and S_e for the enthalpy method are expressed as:

$$S_u = \frac{(1-f_m)^2}{f_m^3 + \varepsilon} A_{\mathrm{mush}} u \tag{6}$$

$$S_v = \frac{(1-f_m)^2}{f_m^3 + \varepsilon} A_{\mathrm{mush}} v + \rho_{\mathrm{PCM}} g \delta (T_{\mathrm{PCM}} - T_{m2}) \tag{7}$$

$$S_e = \rho_{\mathrm{PCM}} L_{\mathrm{PCM}} \frac{\partial f_m}{\partial t} \tag{8}$$

where δ is the thermal expansion coefficient, μ is the dynamic viscosity of the PCM, f_m is the liquid fraction, g is gravitational acceleration, A_{mush} is the mushy zone constant ($A_{\mathrm{mush}} = 10^7$), and ε is a numerical coefficient ($\varepsilon = 10^{-3}$)[3].

4) Enthalpy-porosity method:

The enthalpy-porosity method utilizes liquid fraction to describe the PCM state, assuming that liquid phase ratio equals porosity. The mathematical formulation is expressed as[4]:

$$f_m = \begin{cases} 0 & T \le T_{m1} \\ \dfrac{T - T_{m1}}{T_{m2} - T_{m1}} & T_{m1} < T < T_{m2} \\ 1 & T \ge T_{m2} \end{cases} \tag{9}$$

where T_{m1} and T_{m2} represent solidus and liquidus temperatures respectively.

The energy equation for this method is:

$$\frac{\partial}{\partial t}(\rho_{\mathrm{PCM}} H) + \frac{\partial}{\partial x}(\rho_{\mathrm{PCM}} v_x H) + \frac{\partial}{\partial y}(\rho_{\mathrm{PCM}} v_y H) = \frac{\partial}{\partial x} \left(\lambda \frac{\partial T_{\mathrm{PCM}}}{\partial x} \right) + \frac{\partial}{\partial y} \left(\lambda \frac{\partial T_{\mathrm{PCM}}}{\partial y} \right) \tag{10}$$

where H represents the total enthalpy calculated as the sum of sensible enthalpy h and latent enthalpy ΔH:

$$H = h + \Delta H \tag{11}$$

$$h = h_{\mathrm{ref}} + \int_{T_{\mathrm{ref}}}^{T} c \, dT \tag{12}$$

$$\Delta H = f_m L \tag{13}$$

where h_{ref} is the reference enthalpy, T_{ref} is the reference temperature, and L is the latent heat of fusion.

4.1.2 Initial and Boundary Conditions

1) Initial condition:

$$T_{\text{PCM}}(x,y,0) = T_{\text{fin}}(x,y,0) = T_0, \quad u = v = 0, \quad R_1^2 \le x^2 + y^2 \le R_2^2 \tag{14}$$

2) Boundary conditions:

$$T_{\text{PCM}}(x,y,t) = T_{\text{fin}}(x,y,t) = T_w, \quad u = v = 0, \quad x^2 + y^2 = R_1^2 \tag{15}$$

$$\frac{\partial T(x,y,t)}{\partial n} = 0, \quad u = v = 0, \quad x^2 + y^2 = R_2^2 \tag{16}$$

where T_0 is the initial temperature, T_w is the heat source temperature, R_1 is the radius of the inner wall, and R_2 is the radius of the outer wall.

4.1.3 Numerical Procedure

1) Geometry and material configuration:

Eight fin structures with corresponding interval angles were systematically investigated using ANSYS Fluent. Figure 2 illustrates the cross-section configurations under various interval angles θ. The rectangular fin length (L) was set to 0.018m, and the width (W) was 0.006m. All rectangular fins are uniformly distributed within the cross-section. The interval angle θ varies from 180° to 30°, corresponding to fin numbers from 2 to 12.

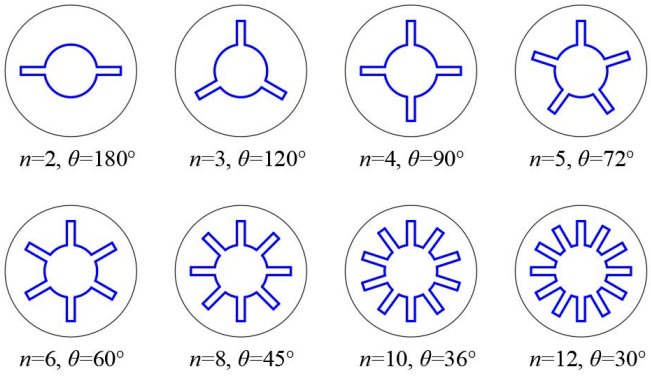

Figure 2: Cross-section configurations under various interval angles θ for rectangular fins

Based on the physical properties of PCM and fin materials along with standard constants, the material parameters utilized in numerical simulations are presented in Table 2.

Table 2: Material property parameters

Symbol	Property	Unit	PCM	Fin (Quartz)
ρ	Density	kg/m^3	780	2650
λ	Thermal conductivity	W/(m·K)	0.15	214
c	Specific heat	J/(kg·K)	2000	-
T_{m1}	Solidus temperature	K	273.15	-
T_{m2}	Liquidus temperature	K	333	-
δ	Thermal expansion coefficient	K^{-1}	0.0002	-
μ	Dynamic viscosity	kg/(m·s)	/	-

2) Mesh size and time step validation:

To validate the finite element model, grid and time step independence studies were conducted. Temperature evolution was compared across different mesh densities to identify accurate and efficient grid configurations, while phase change time comparisons were performed to determine optimal time steps.

Mesh size validation:

The mesh size was varied from 4mm (Grid1) to 1mm (Grid4) to establish the accurate and efficient grid configuration. Figure 3 illustrates the meshing conditions under different mesh sizes. Figure 4(a) presents the PCM temperature evolution for four grid conditions. With mesh refinement from Grid3 to Grid4, temperature evolution curves become virtually identical. Considering computational efficiency, a mesh size of 2mm was selected for this study.

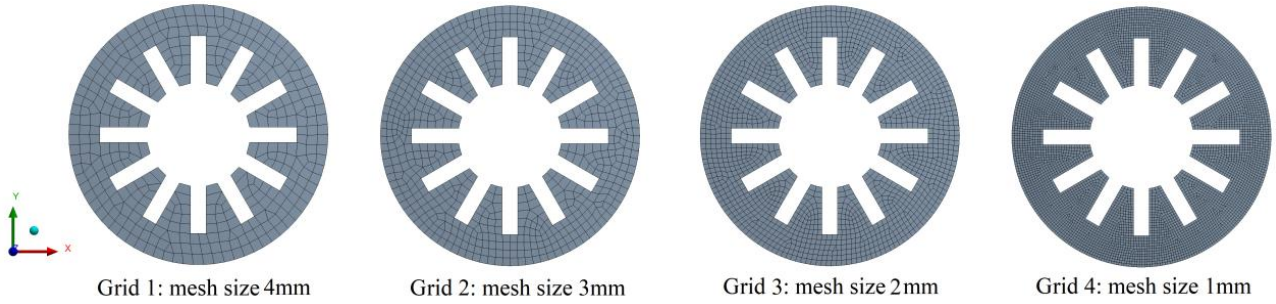

Figure 3: Meshing conditions under different mesh sizes

Time step validation:

Figure 4(b) demonstrates that phase change time results stabilize at time steps of 1s or 2s. Consequently, a time step of 2s was selected for the simulations.

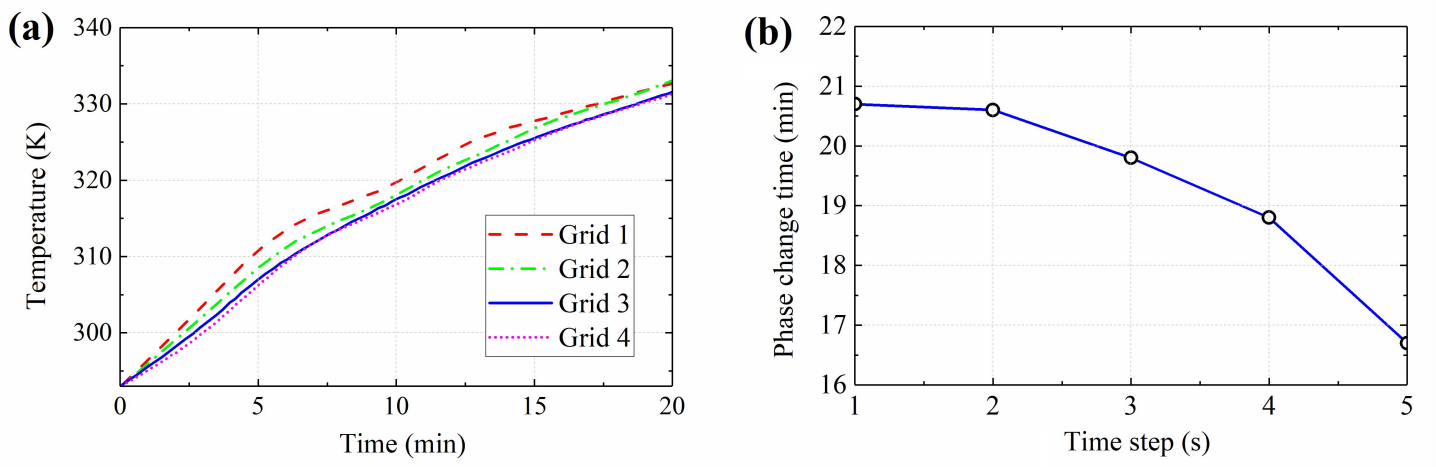

Figure 4: Validation of the numerical model: (a) grid independence test, (b) time step independence test

4.2 Heat Transfer Process Analysis

4.2.1 Temperature Distribution

Figure 5 presents the temperature distribution evolution considering the influence of fin interval angle. The phase change initiates in regions surrounding the fins due to heat conduction from fins to the PCM. As time progresses, the phase change front extends toward the outer shell through fin-mediated heat transfer, ultimately causing complete melting of the PCM within the tank to liquid phase.

The incorporation of fins significantly accelerates the PCM phase change process. This acceleration occurs because reduced inter-fin spacing enables the internal PCM to rapidly absorb thermal energy and melt under the dominant heat

conduction mechanism.

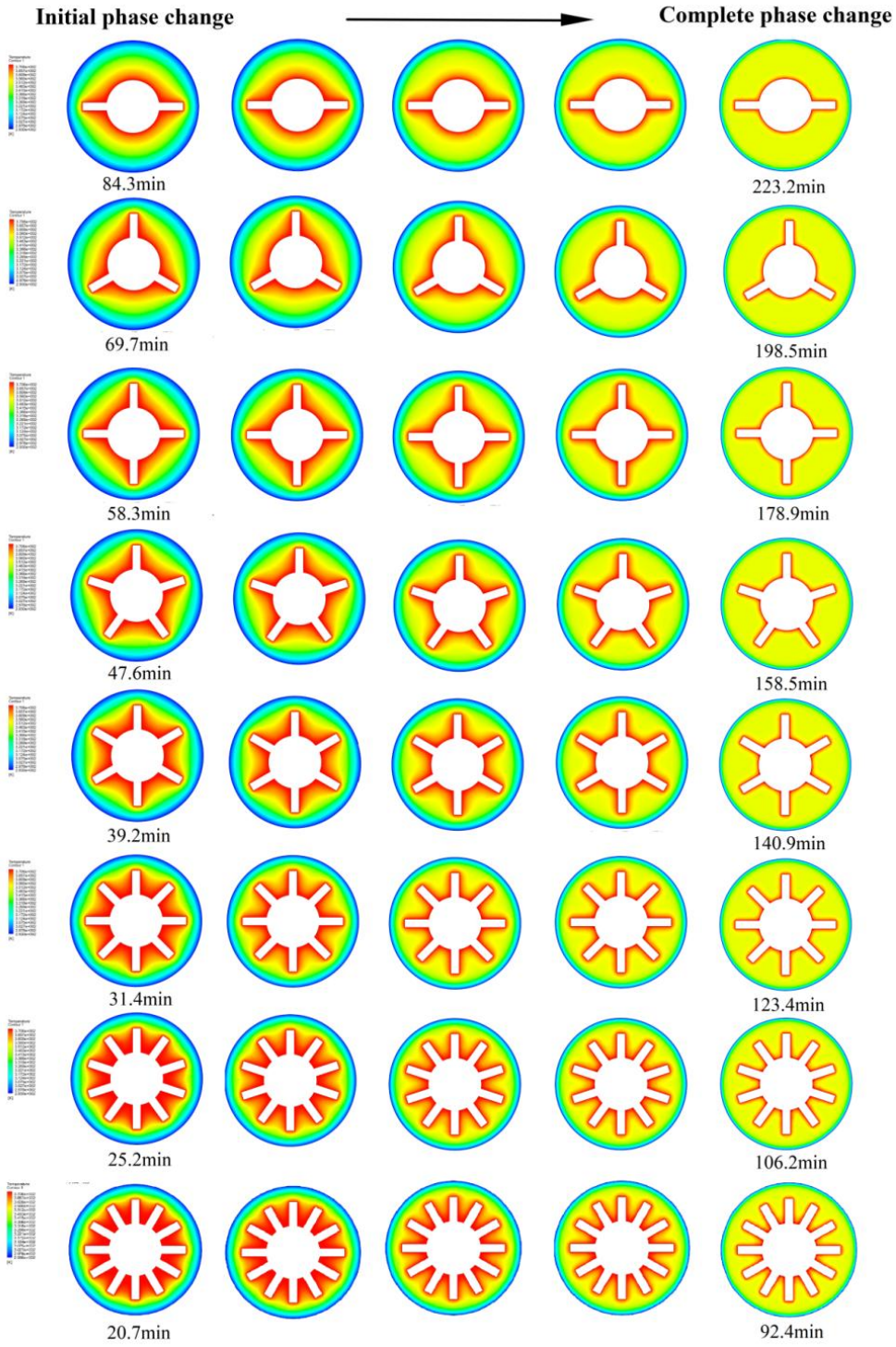

Figure 5: Temperature distribution evolution under different interval angles θ for rectangular fins

4.2.2 Phase Change Time

Both initial and complete phase change times for PCM melting were systematically investigated:

- **Initial phase change time** t_1: The time required for PCM to transition from room temperature (293K) to the phase change temperature (333K).
- **Complete phase change time** t_2: The time at which the PCM completely transforms to liquid phase.

Figure 6 illustrates the variation of phase change time under different interval angles. Both initial and complete phase change times decrease with decreasing interval angle (θ) or equivalently increasing fin number (n). When θ decreases from 60°, 45°, 36°, to 30°, the initial phase change times are 39.2min, 31.4min, 25.2min, and 20.7min respectively.

However, as θ decreases, fin efficiency exhibits slight reduction. This phenomenon results from the contraction of spacing in the tank's central region, causing a marginal reduction in PCM volume. Therefore, optimal interval angle determination requires consideration of competing effects to achieve maximum heat transfer efficiency.

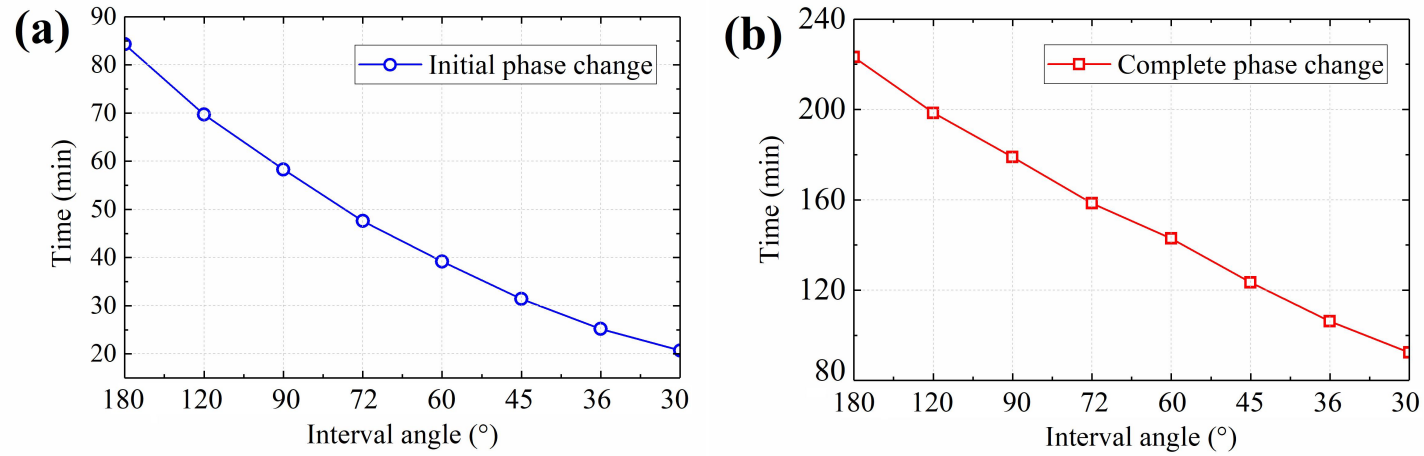

Figure 6: Phase change time under different configurations: (a) initial phase change time, (b) complete phase change time

Table 3: Phase change time for different fin configurations

Fin number	Interval angle (°)	Initial phase change time (min)	Complete phase change (min)
2	180	84.3	92.4
3	120	69.7	106.2
4	90	58.3	123.4
5	72	47.6	142.9
6	60	39.2	158.5
8	45	31.4	178.9
10	36	25.2	198.5
12	30	20.7	223.2

4.2.3 Liquid Fraction Analysis

Figure 7 presents the variation of liquid fraction over time under different fin numbers. The PCM melting rate increases with increasing fin number n. However, with continued increase in n (or equivalently decrease in θ), the melting rate growth begins to diminish beyond a critical threshold value. The underlying mechanism for this phenomenon is explained in Section 4.3. Therefore, the interval angle θ must be matched with overall fin arrangement to achieve optimal thermal efficiency.

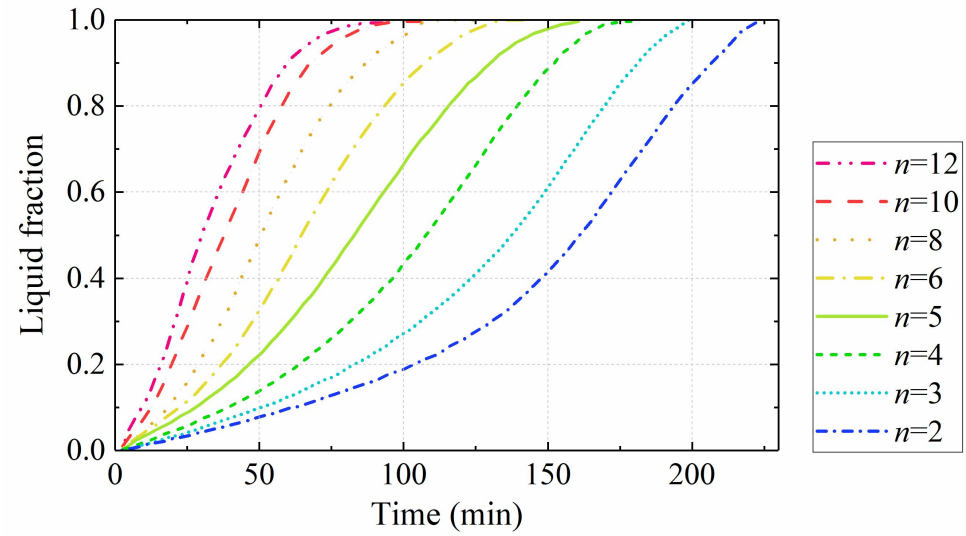

Figure 7: Liquid fraction evolution under different fin configurations

4.3 Optimization of the Interval Angle

4.3.1 Enhancement Ratio

To identify the optimal interval angle, numerical simulations were extended to $n = 20$ fins to further investigate the effect of fin number on heat transfer performance. The enhancement ratio E_R was proposed to quantitatively evaluate the influence of n on heat transfer rate:

$$E_R = \frac{f_m(n) - f_m(\text{no fins})}{f_m(\text{no fins})} \times 100\% \tag{17}$$

where $f_m(n)$ is the liquid fraction of PCM under the n-fin configuration, and $f_m(\text{no fins})$ is the liquid fraction without fins.

Figure 8(a) illustrates the enhancement effects of different n on PCM melting rate. With increasing n, the enhancement ratio E_R increases significantly, indicating that larger fin numbers provide greater advantages during the immediate stage following heating initiation. **The E_R of 15 fins ($\theta = 24$) achieves the maximum value of 12.3%**, while E_R values for 14 and 16 fins are relatively lower at 12.1% and 11.8% respectively.

However, with further increase in n, particularly when n exceeds 15, E_R gradually decreases as excessive fins begin to impede heat transfer, causing PCM near the wall surface to melt with difficulty. This occurs because reduced spacing between adjacent fins increases flow resistance, thereby weakening fluid movement amplitude[5].

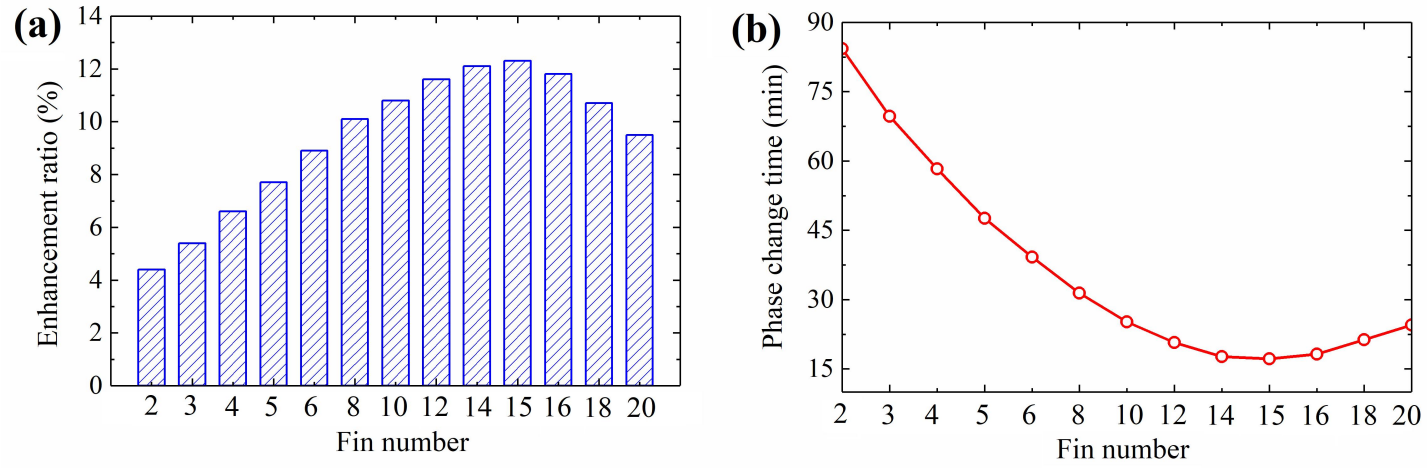

Figure 8: Comparison of different fin numbers n: (a) enhancement ratio, (b) initial phase change time

4.3.2 Phase Change Time Analysis

The initial phase change times under different fin numbers were calculated correspondingly. Figure 8(b) presents the initial phase change time for various n values. The results confirm that the **15-fin structure ($\theta = 24$) exhibits optimal heating capacity with minimum melting time of 17.2 min**.

5 Task 2: Parametric Study of Triangular Fins

5.1 Effect of the Interval Angle

5.1.1 Influence on Heat Transfer

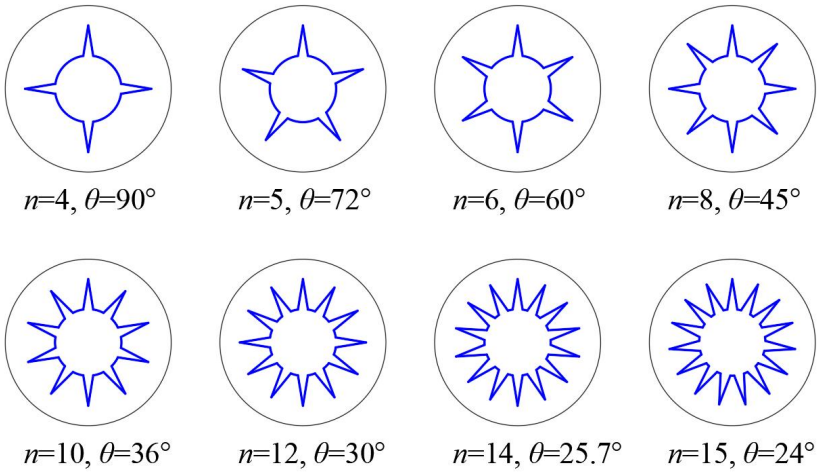

Figure 9: Cross-section configurations under various interval angles θ for triangular fins

Following the methodology established for rectangular fins, eight interval angles of triangular fins were systematically investigated. Figure 9 depicts the cross-section configurations under various θ values. The fin length L was 0.018m and width W was 0.006m. The interval angle θ varies from 90° to 24°, corresponding to fin numbers from 4 to 15.

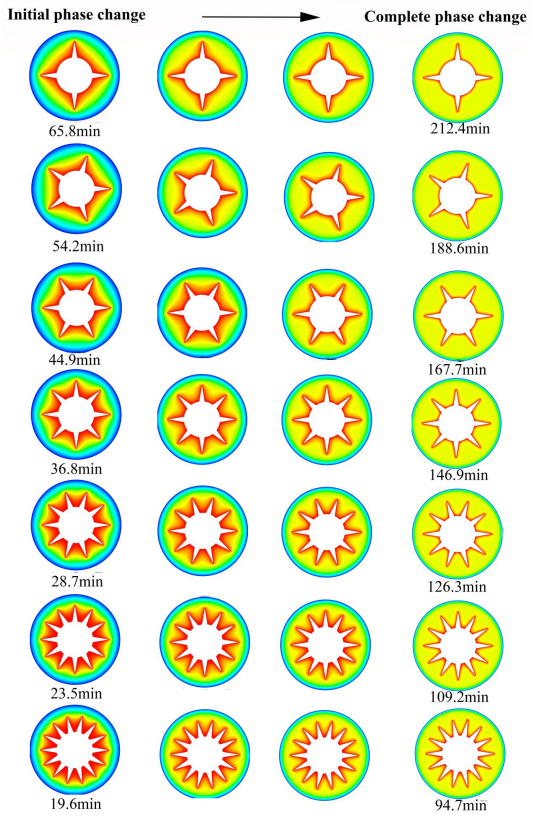

Figure 10: Temperature distribution evolution under different interval angles θ for triangular fins

Heat storage performance was analyzed by varying θ while maintaining other parameters constant. Figure 10 presents the temperature distribution under various θ for triangular fins. The melting time decreases by approximately

44.1% when θ changes from 90° ($n = 4$) to 45° ($n = 8$), and significantly decreases by approximately 70.2% when θ equals 24° ($n = 15$). This improvement results from enhanced natural convection and reduced thermal resistance, both attributable to the larger heat transfer surface area provided by increased fin numbers.

Greater fin quantities accelerate the heating process within the system. However, the rate of phase change time reduction diminishes when θ exceeds 36° ($n = 10$), particularly for 26° ($n = 14$) and 24° ($n = 15$). This behavior occurs because fin presence interferes with natural convection flow patterns. Consequently, limits exist for both interval angle θ and fin number n.

5.1.2 Comparison with Rectangular Fins

Figure 11(a) compares phase change times between rectangular and triangular fins for various θ values. Heating efficiency progressively improves as the fin design evolves from triangular to rectangular configurations. At $\theta = 24$, improvements of 63.83%, 56.34%, 46.73%, 46.73%, and 31.71% in heating rate are achieved compared with $\theta = 72$, 60°, 45°, 36°, and 30° respectively. Notably, the time difference between both fin types decreases as θ decreases.

Figure 11(b) presents liquid fraction curves for different θ during melting. Rectangular fins demonstrate superior efficiency compared to triangular fins by reducing total melting time. For θ values of 72°, 45°, 30°, and 24°, rectangular fins improve heating efficiency by 11.95%, 14.99%, 15.43%, and 15.62% respectively compared with triangular fins. The improved heating efficiency represents the relative difference in melting time between both configurations at identical θ.

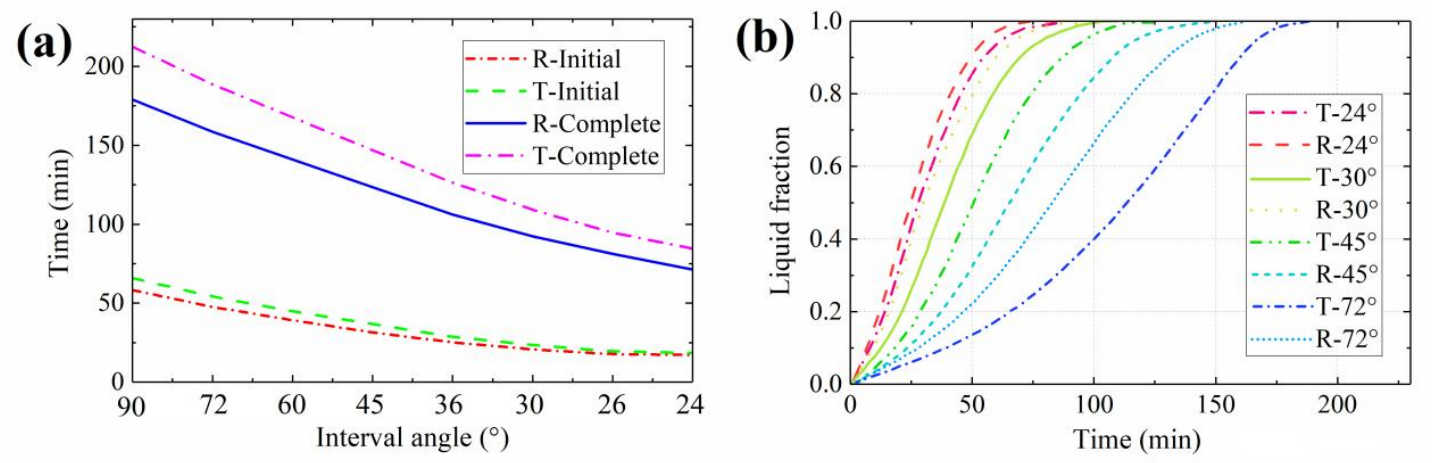

Figure 11: Comparison between rectangular fins and triangular fins for various θ: (a) phase change time, (b) liquid fraction evolution (T: triangular fins; R: rectangular fins)

5.2 Effect of the Fin Length

Four fin lengths (L) ranging from 24mm to 6mm were investigated. Figure 12 illustrates the cross-section configurations under various L values. The interval angle θ was maintained at 30° and the width W was 6mm.

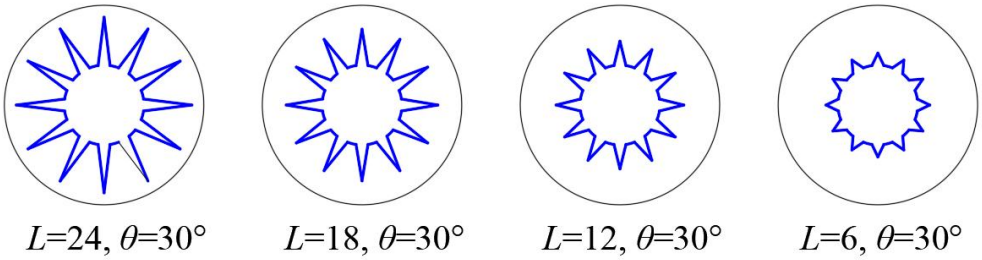

Figure 12: Cross-section configurations under various fin lengths L

5.2.1 Influence on Heat Transfer

Figure 13 presents the temperature distribution under various fin lengths for triangular fins. The melting time decreases by approximately 70.4% when L increases from 12mm to 24mm.

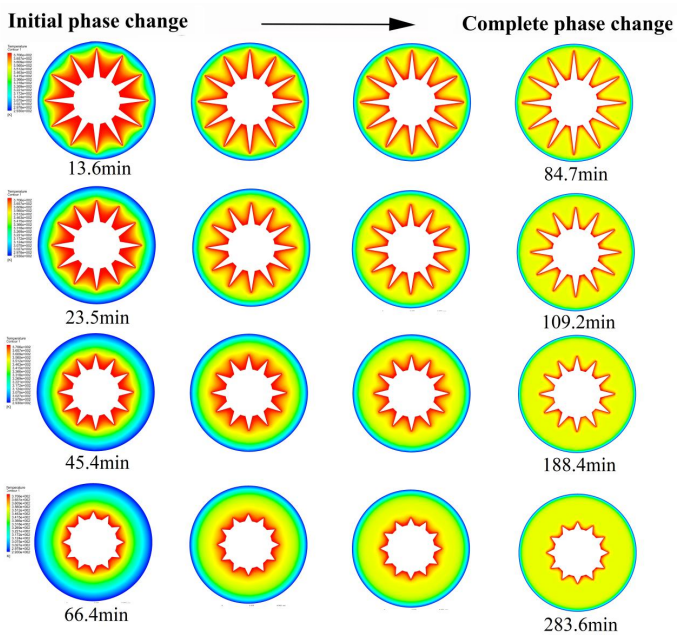

Figure 13: Temperature distribution evolution under different fin lengths L

5.2.2 Comparison and Analysis

Figure 14(a) compares phase change times between different fin lengths. Heating efficiency improves significantly with increasing L. **For initial phase change**, $L = 24$mm achieves 79.52%, 70.04%, and 42.17% heating rate improvements compared with $L = 6$mm, 12mm, and 18mm respectively. **For complete phase change**, melting time with $L = 24$mm is reduced by 70.13%, 55.06%, and 22.43% compared with $L = 6$mm, 12mm, and 18mm respectively.

Figure 14(b) presents liquid fraction curves for different L during melting, confirming that longer fins achieve greater efficiency than shorter fins.

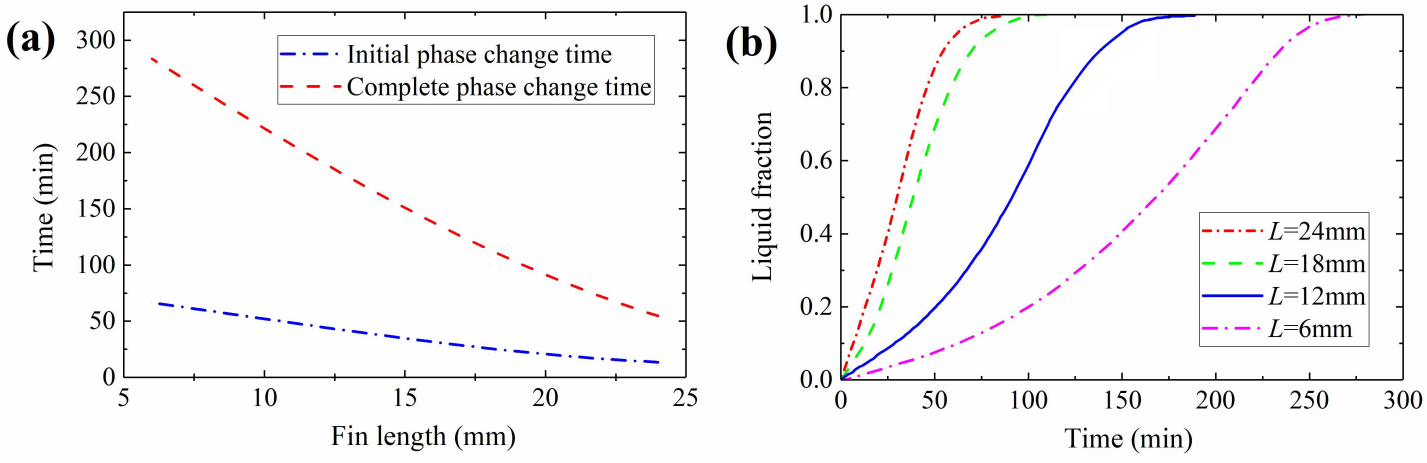

Figure 14: Comparison between various fin lengths L: (a) phase change time, (b) liquid fraction evolution

5.3 Effect of the Fin Width

Four fin widths (W) ranging from 10mm to 4mm were investigated. Figure 15 depicts the cross-section configurations under various W values. The interval angle θ was maintained at 30° and the length L was 18mm.

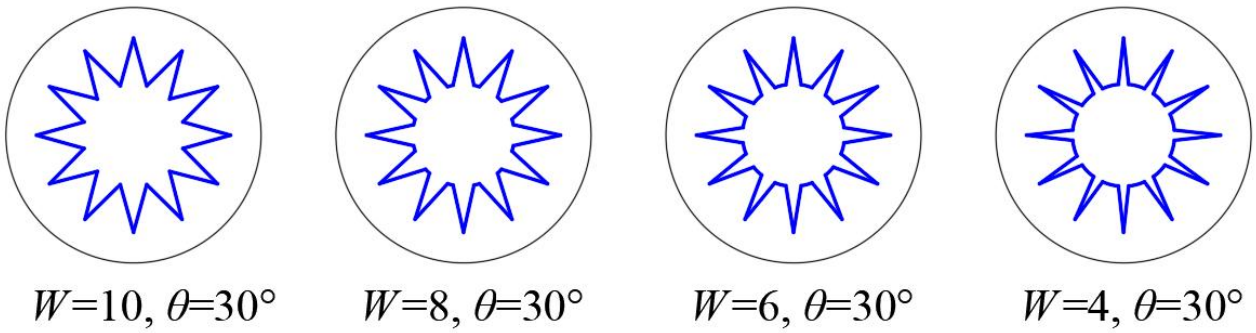

Figure 15: Cross-section configurations under various fin widths W

5.3.1 Influence on Heat Transfer

Figure 16 presents the temperature distribution under various fin widths for triangular fins. The melting time decreases by approximately 35.9% when W increases from 4mm to 10mm. Therefore, larger fin widths provide moderate improvement to the heat transfer process.

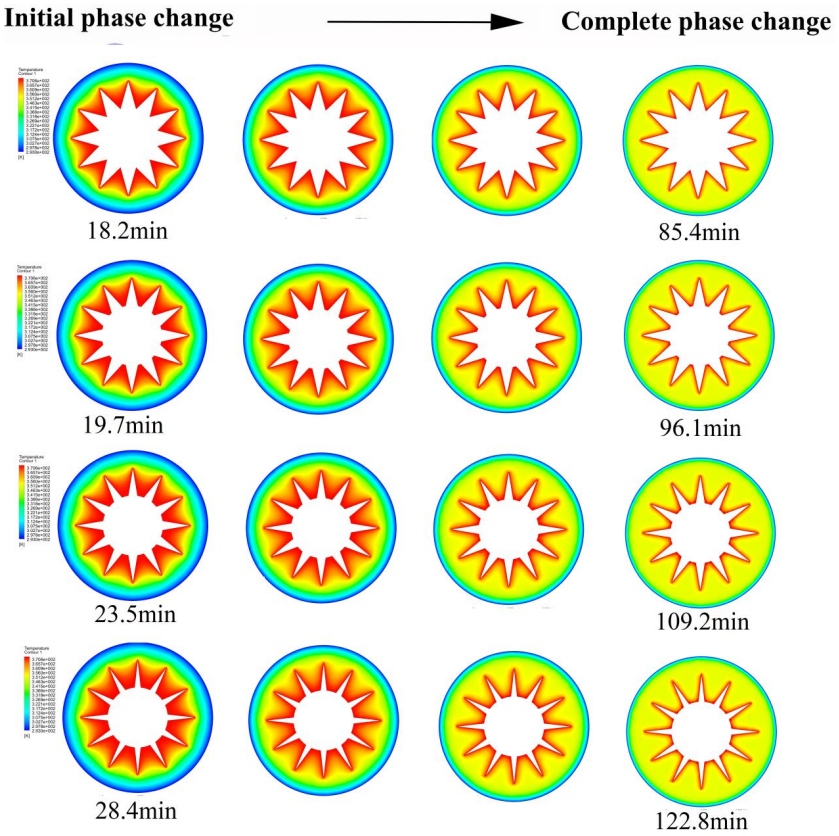

Figure 16: Temperature distribution evolution under different fin widths W

5.3.2 Comparison and Analysis

Figure 17(a) compares phase change times between different fin widths. Heating efficiency improves moderately with increasing W. **For initial phase change**, $W = 10$mm achieves 7.6%, 22.55%, and 35.91% heating rate improvements compared with $W = 8$mm, 6mm, and 4mm respectively. **For complete phase change**, melting time with $W = 10$mm is reduced by 11.13%, 21.79%, and 30.52% compared with $W = 8$mm, 6mm, and 4mm respectively.

Figure 17(b) presents liquid fraction curves for different W during melting, demonstrating that wider fins achieve greater efficiency than narrower fins.

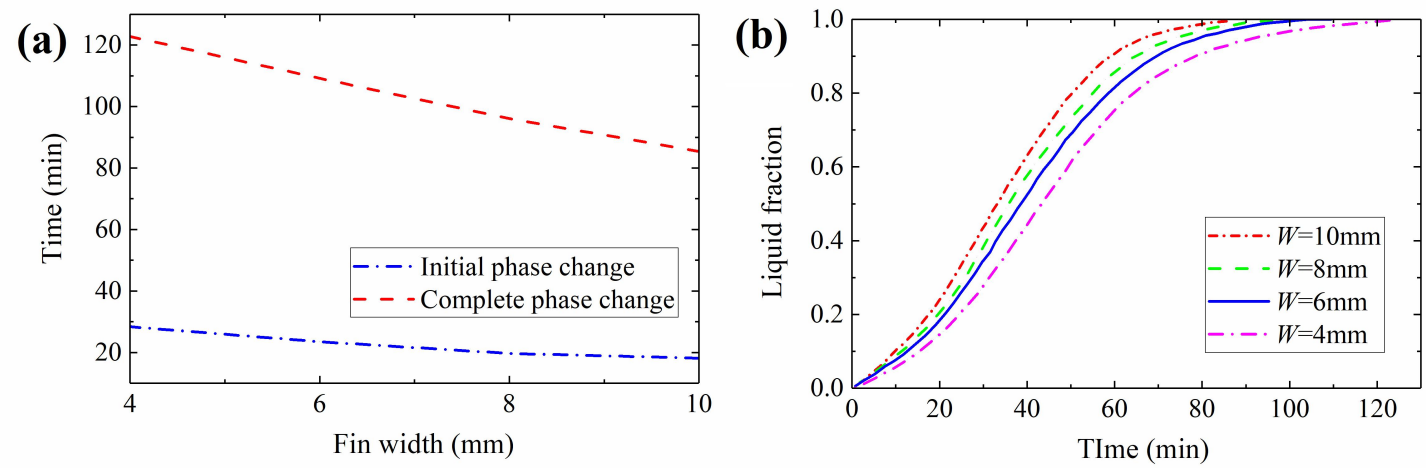

Figure 17: Comparison between various fin widths W: (a) phase change time, (b) liquid fraction evolution

6 Task 3: Optimal Fin Distribution via Topology and Fractal Optimization

Optimal spatial distribution of fins is required to achieve maximum heat transfer capacity. The optimization objective is to maximize heat transfer efficiency within the heat storage tank.

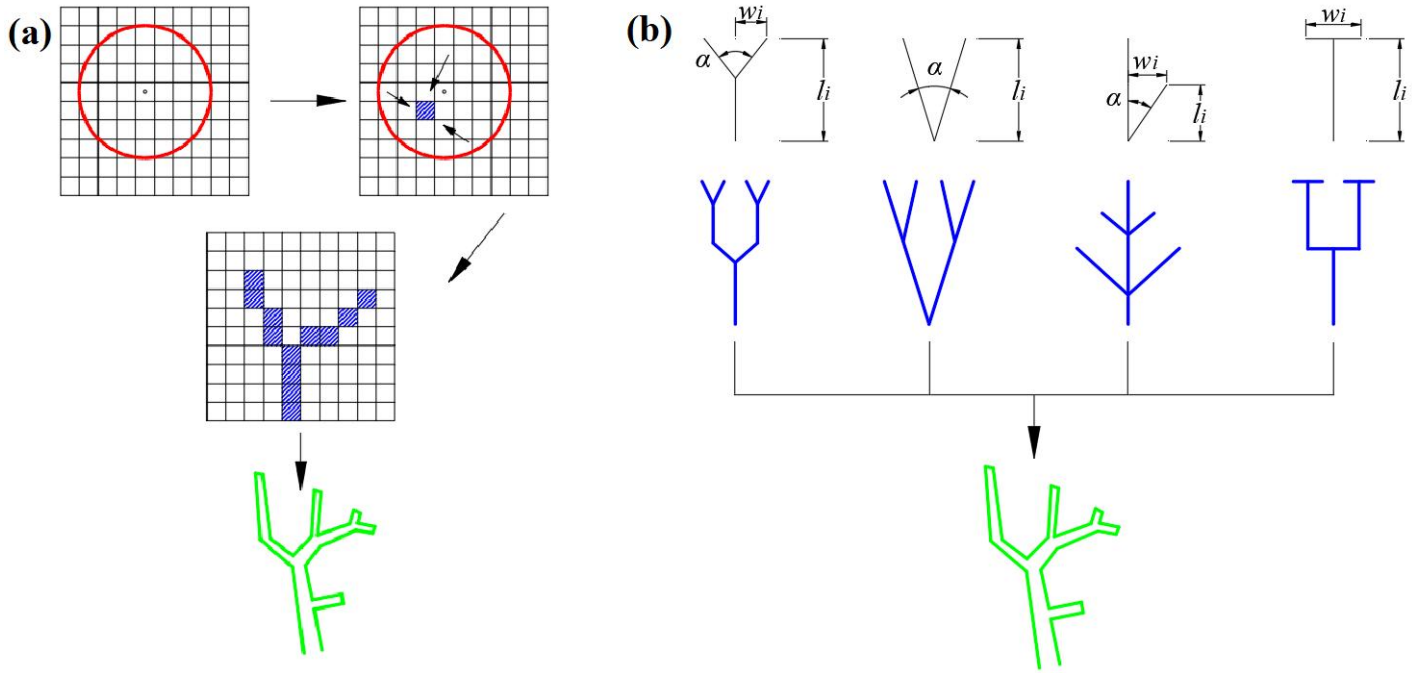

Figure 18: Principle diagrams of fin optimization methodologies: (a) topology optimization, (b) fractal design

6.1 Topology Optimization

6.1.1 Variable Density Method

To identify optimal fin configurations, the variable density method (VDM)[6] is employed to optimize fin spatial distribution. The VDM introduces materials with variable imaginary density and corresponding physical properties, transforming structural optimization problems into material layout problems.

The interpolation relationships between original and imaginary variables are defined by the Solid Isotropic Material with Penalization (SIMP) method[6]:

$$K(x) = (K_{\max} - K_{\min})(S)^{\eta} + K_{\min} \tag{18}$$

$$\rho(x) = (\rho_{\max} - \rho_{\min})S + \rho_{\min} \tag{19}$$

$$\lambda(x) = (\lambda_{\max} - \lambda_{\min})S + \lambda_{\min} \tag{20}$$

$$L(x) = (1 - S)L \tag{21}$$

$$\theta(x) = (\theta_{\max} - \theta_{\min})S + \theta_{\min} \tag{22}$$

where $K(x)$ is element stiffness, $\rho(x)$, $\lambda(x)$, $L(x)$, and $\theta(x)$ represent pseudo-thermal density, conductivity, fin length, and interval angle respectively, ρ_e is pseudo density with $S \in [0,1]$, and η is the penalty factor with $\eta \in [0,1]$.

The topology optimization solution procedure based on VDM is expressed as:

$$\begin{aligned} &\text{find} \quad \boldsymbol{\rho} = (\rho_1, \rho_2, \cdots, \rho_N) \\ &\text{minimize} \quad C = \frac{1}{N}\sum_{i=1}^{N} T_i = T \\ &\text{subject to:} \\ &\quad \frac{1}{V}\sum_{i=1}^{N} V_i \le f \\ &\quad \rho_{\min} \le \rho_i \le 1 \end{aligned} \tag{23}$$

where T is the temperature array, K is the stiffness matrix, Q is the heat load matrix, N is the number of elements, V_i is the volume of element i, f is the maximum volume fraction, and $\rho_{\min}$ is the minimum pseudo density (set to 10^{-9} to prevent singular matrix occurrence).

The Lagrangian function for this approximate mathematical programming problem with continuous variables is[7]:

$$L = C + \Lambda_1(V - fV_0) + \Lambda_2\sum_{i=1}^{N}(\rho_i - \rho_i^u) + \Lambda_3\sum_{i=1}^{N}(\rho_i - \rho_i^l) + \Lambda_4\sum_{i=1}^{N}(\rho_i^l - \rho_i) \tag{24}$$

According to the Kuhn-Tucker optimality conditions[7], at the optimum point:

$$K(x)\frac{\partial V_0 T_i}{\partial \rho_i} = \Lambda_1 \tag{25}$$

6.1.2 Optimization Result

The optimization objective was maximum average temperature within the PCM region. ANSYS finite element software was employed for steady-state solutions. The method of moving asymptotes (MMA)[8] was utilized for calculations. Optimization tolerance was set to 10^{-3}. The initial element pseudo density ρ_e was 0.5, and maximum volume fraction f was set to 0.05. To obtain explicit topology results, the penalty factor η was set to 3.

The maximum average temperature achieved was 362.46K after 500 iterations in MATLAB. The fin distribution obtained through topology optimization is shown in Figure 19(a).

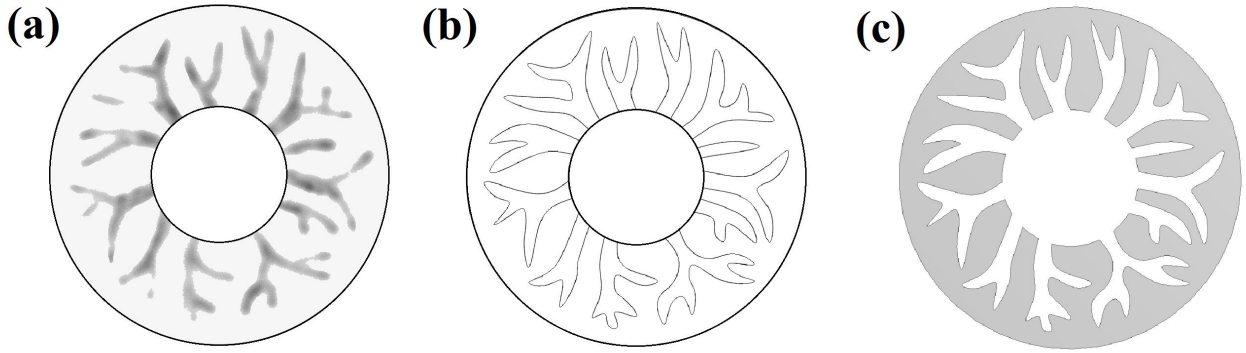

Figure 19: Topological shape of fin distribution: (a) topology optimization result, (b) image reconstruction, (c) final fin distribution for ANSYS simulation

6.1.3 Image Reconstruction

Although SIMP effectively reduces intermediate density values, junctions between fins and PCM regions persist due to design variable continuity. Therefore, fuzzy boundaries require filtering to sharpen the topological representation. The reconstruction function is[9]:

$$x_p = \frac{\tanh(\beta\mu) + \tanh(\beta(x_f - \mu))}{\tanh(\beta\mu) + \tanh(\beta(1-\mu))} \tag{26}$$

where x_p is the mapped variable, x_f is the initial variable, μ is the mapping threshold (set to 0.5), and β is the control parameter for mapping steepness.

6.1.4 Heat Transfer Process

Figure 20 presents the temperature distribution for topology-optimized fins. Figure 21 shows the phase change time and liquid fraction respectively. Compared with rectangular and triangular fins, topology-optimized fins achieve significantly improved heating efficiency.

For initial phase change, topology-optimized fins achieve 18.84% and 28.17% heating rate improvements compared with rectangular and triangular fins respectively. **For complete phase change**, melting time with topology-optimized fins is reduced by 41.81% and 50.82% compared with rectangular and triangular fins respectively.

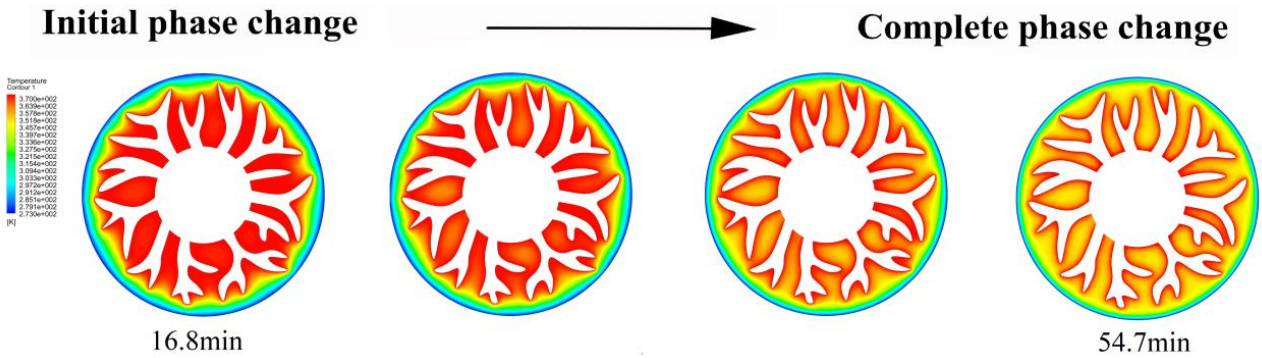

Figure 20: Temperature distribution evolution for topology-optimized fin configuration

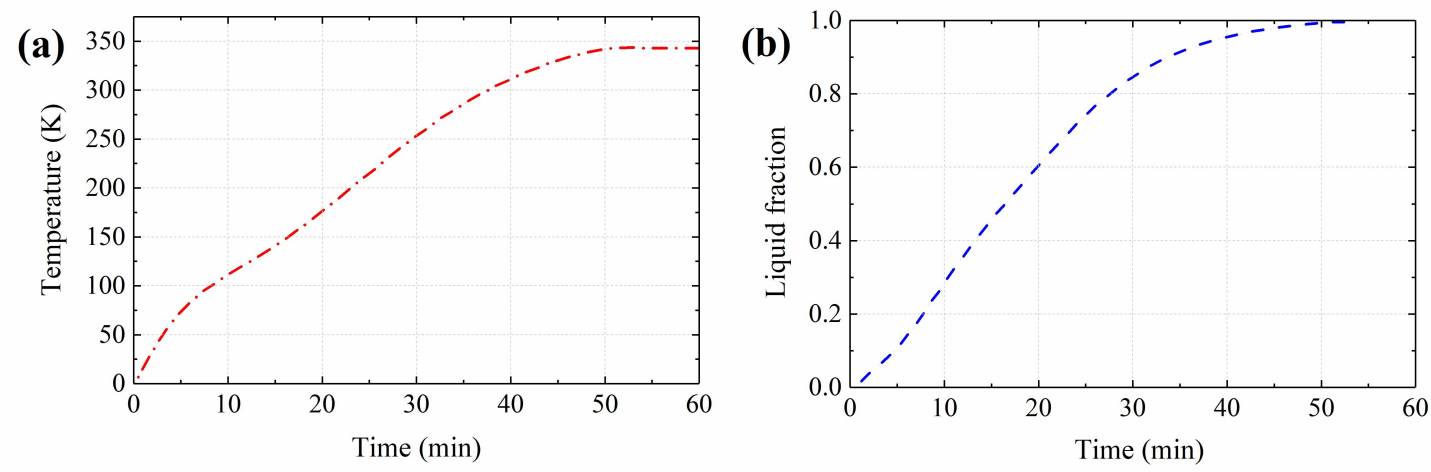

Figure 21: Temperature variation and liquid fraction for topology-optimized fins

6.2 Fractal Optimization

6.2.1 Murray's Law and Branch Generation

Fractal fins possess statistically self-similar and multi-scale properties, which can be considered as biomimetic analogues of fern leaf structures. The optimal transportation in leaf venation is derived from Murray's law[10]:

$$r^3 = r_1^3 + r_2^3 + r_3^3 + \cdots + r_n^3 \tag{27}$$

where r is the parent tube radius, and r_1 to r_n are the branching tube radii.

Based on coupling with Murray's law, the generalized Murray's law is obtained as[11]:

$$r^\gamma = \frac{1}{1-\zeta}\sum_i^n r_i^\gamma \tag{28}$$

where ζ is the heat variation ratio, γ is determined by the transport mode, and n is the number of child tubes.

The fern-fractal fins were designed using generalized Murray's law. The exponent γ was set to 2 to achieve optimal thermal performance for heat conduction[12]. The fractal fin construction procedure is as follows:

Step 1: The parent branch generates two children branches. The parent branch length and width are L_0 and W_0 respectively. The first-level children branches have length L_1 and width W_1, while second-level branches have L_2 and W_2. The children branch terminals are positioned close to the outer tank wall.

Step 2: The lengths of any two successive branches (parent branch and middle child branch) satisfy:

$$L_i = \alpha L_{i-1} \tag{29}$$

where α is the length ratio reflecting the length growth law.

Step 3: The correlation between parent and children branch widths is:

$$W_i^2 = \frac{1}{3}W_{i-1}^2 \tag{30}$$

Step 4: Repeat Steps 1-3 until the desired branching level is achieved. The number of branches per level is optimized by maximizing average temperature. The fern-fractal fins are thus constructed.

6.2.2 Optimization Result

The fin distribution obtained through fractal design is shown in Figure 22(a). The fin distribution used in ANSYS is depicted in Figure 22(b).

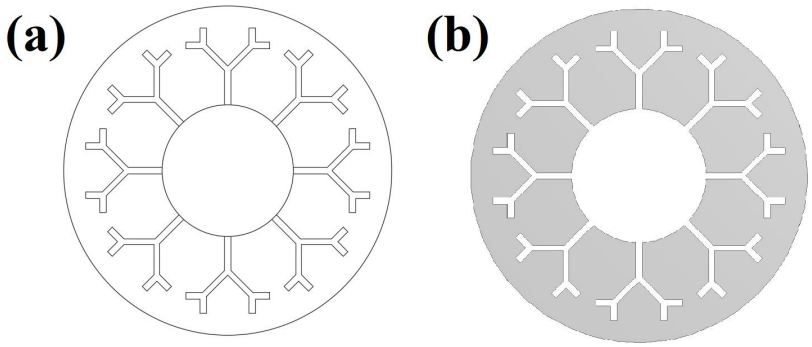

Figure 22: Fractal shape of fin distribution: (a) fractal optimization result, (b) final fin distribution for ANSYS simulation

6.2.3 Heat Transfer Process

For initial phase change, fractal-optimized fins achieve 14.01% and 24.25% heating rate improvements compared with rectangular and triangular fins respectively. **For complete phase change**, melting time with fractal-optimized fins is reduced by 32.57% and 42.94% compared with rectangular and triangular fins respectively.

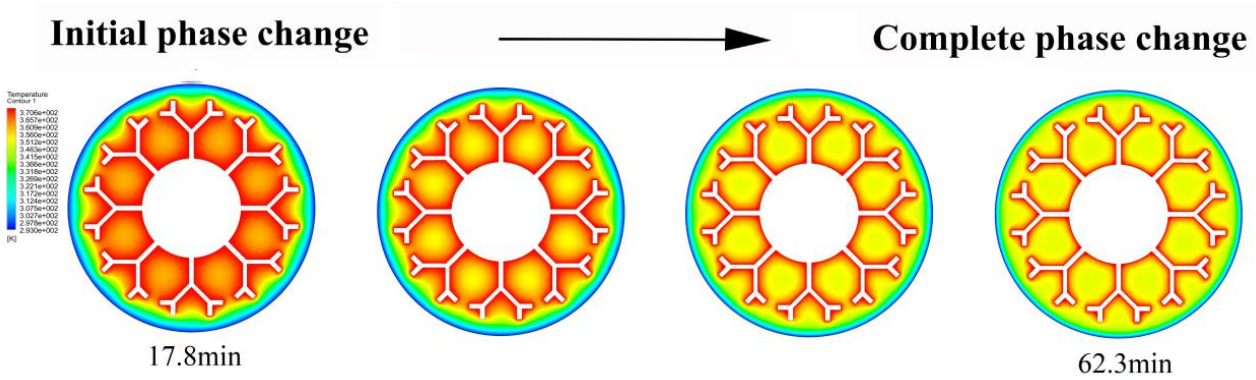

Figure 23: Temperature distribution evolution for fractal-optimized fin configuration

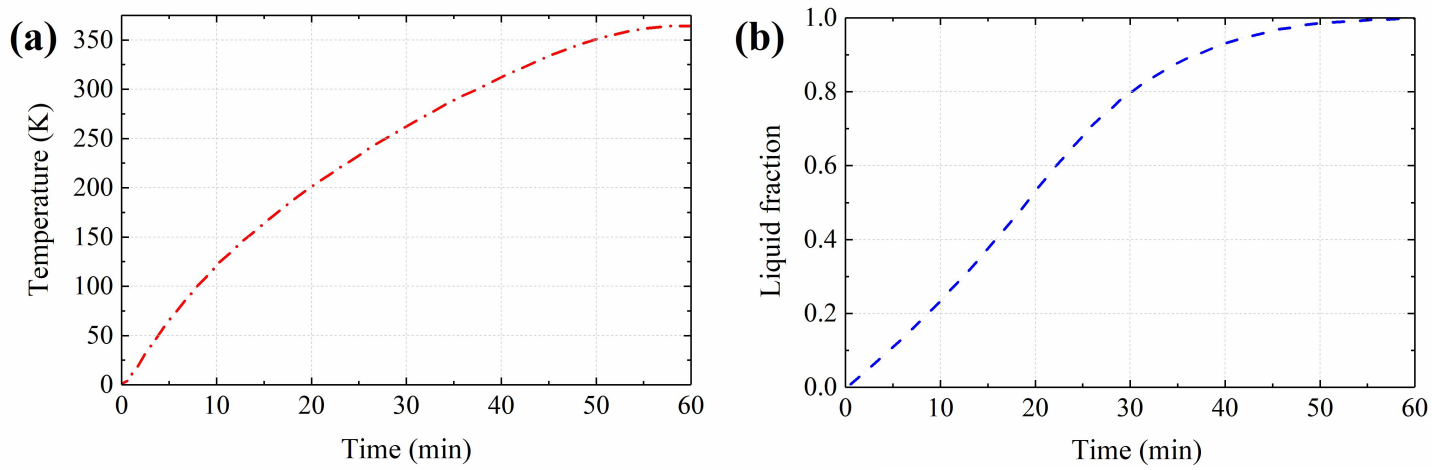

Figure 24: Temperature variation and liquid fraction for fractal-optimized fins

6.3 Comparison of Two Optimization Models

Based on phase change time and liquid fraction analyses, topology optimization demonstrates superior efficiency compared to fractal optimization. **For initial and complete phase change times, topology optimization achieves 5.61% and 13.8% heating rate improvements compared with fractal optimization.**

7 Sensitivity Analysis

The sensitivity of exponent γ in the fractal model was evaluated by varying the fractal dimension and comparing performance across different configurations. The exponent γ was set to 2, 3, and 4 respectively to investigate the influence of fractal dimension (i.e., bifurcation number) on temperature evolution.

Figure 25 presents temperature variation comparisons under different γ values. The temperature rise is slightly enhanced with increasing γ. This occurs because no significant change in spacing between adjacent fins results from the fractal tree control mechanism. Therefore, the fractal design model demonstrates good stability.

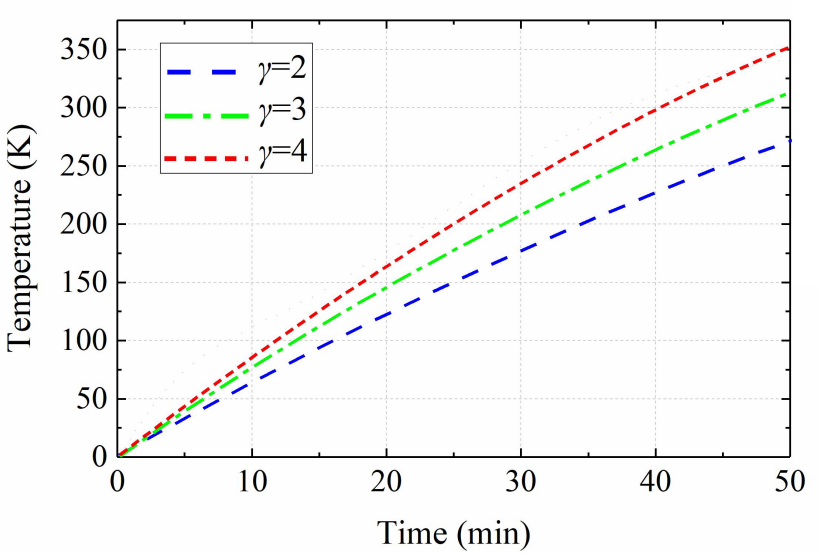

Figure 25: Comparison of temperature variation under different fractal exponent γ values

8 Strengths and Weaknesses

8.1 Strengths

- **Model Stability:** When predicting optimal fin distribution, the fractal optimization model demonstrates superior stability compared to topology optimization. However, topology optimization yields fin distributions with higher heat transfer efficiency.
- **Multi-objective Consideration:** The topology optimization model identifies optimal fin distributions while considering various balancing factors in optimization objectives.
- **Automated Optimization:** The proposed topology and fractal optimization models can automatically search for optimal fin distributions under objective function constraints.
- **Comprehensive Parametric Analysis:** Systematic investigation of fin parameters (interval angle, length, width) provides quantitative design guidelines.
- **Practical Applicability:** The fractal optimization approach generates regular fin structures suitable for manufacturing processes.

8.2 Weaknesses

- **Manufacturing Complexity:** The optimized shapes obtained through topology optimization are typically irregular, causing difficulties in manufacturing and practical application of fin components.
- **Objective Function Sensitivity:** Objective function selection critically influences final optimization results. Different optimization objectives may yield different optimal configurations, requiring careful objective function specification for specific applications.
- **Two-dimensional Limitation:** Current models are limited to two-dimensional analysis, which may not fully capture three-dimensional flow and heat transfer phenomena in actual storage tanks.

References

[1] Voller V R, Brent A D, Prakash C. Modelling the mushy region in a binary alloy[J]. Applied Mathematical Modelling, 1990, 14(6): 320-326.

[2] Cheng Y, Xuan Z B, Xi C B, et al. Melting performance enhancement of a latent heat storage unit using gradient fins[J]. International Journal of Heat and Mass Transfer, 2022, 150: 119300.

[3] Al-Abidi A A, Mat S, Sopian K, et al. Internal and external fin heat transfer enhancement technique for latent heat thermal energy storage in triplex tube heat exchangers[J]. Applied Thermal Engineering, 2013, 53(1): 147-156.

[4] Seddegh S, Joybari M M, Wang X, et al. Experimental and numerical characterization of natural convection in a vertical shell-and-tube latent thermal energy storage system[J]. Sustainable Cities and Society, 2017, 35: 13-24.

[5] Mao Q, Li Y, Li G, et al. Study on the influence of tank structure and fin configuration on heat transfer performance of phase change thermal storage system[J]. Energy, 2021, 235: 121382.

[6] Bendsøe M P, Sigmund O. Material interpolation schemes in topology optimization[J]. Archive of Applied Mechanics, 1999, 69(9-10): 635-654.

[7] Geng D C. Basis of Engineering Structure Optimization Design[M]. Dalian: Dalian University of Technology Press, 2012.

[8] Svanberg K. The method of moving asymptotes—a new method for structural optimization[J]. International Journal for Numerical Methods in Engineering, 1987, 24(2): 359-373.

[9] Zhao C Y, Wu Z G. Heat transfer enhancement of high temperature thermal energy storage using metal foams and expanded graphite[J]. Solar Energy Materials and Solar Cells, 2011, 95(2): 636-643.

[10] McCulloh K A, Sperry J S, Adler F R. Water transport in plants obeys Murray's law[J]. Nature, 2003, 421(6926): 939-942.

[11] Zhou Y, Kassab G S, Molloi S. On the design of the coronary arterial tree: a generalization of Murray's law[J]. Physics in Medicine and Biology, 1999, 44(12): 2929-2945.

[12] Xu P, Sasmito A P, Yu B, et al. Transport phenomena and properties in treelike networks[J]. Applied Mechanics Reviews, 2016, 68(4): 040802.

Appendices

Appendix A: MATLAB Code for Topology Optimization

```
function TopologyOptimization_HeatFin()
    nelx = 100;
    nely = 100;
    volfrac = 0.05;
    penal = 3;
    rmin = 2.5;
    ft = 1;

    x = repmat(volfrac, nely, nelx);
    xPhys = x;
    loop = 0;
    change = 1;
    maxloop = 500;
```

```
    tolx = 1e-3;

    [KE] = lk_heat;
    nodenrs = reshape(1:(1+nelx)*(1+nely), 1+nely, 1+nelx);
    edofVec = reshape(nodenrs(1:end-1, 1:end-1)+1, nelx*nely, 1);
    edofMat = repmat(edofVec, 1, 4) + repmat([0 nely+[1 0] -1], nelx*nely, 1);
    iK = reshape(kron(edofMat, ones(4,1))', 16*nelx*nely, 1);
    jK = reshape(kron(edofMat, ones(1,4))', 16*nelx*nely, 1);

    iH = ones(nelx*nely*(2*(ceil(rmin)-1)+1)^2, 1);
    jH = ones(size(iH));
    sH = zeros(size(iH));
    k = 0;
    for i1 = 1:nelx
        for j1 = 1:nely
            e1 = (i1-1)*nely+j1;
            for i2 = max(i1-(ceil(rmin)-1),1):min(i1+(ceil(rmin)-1),nelx)
                for j2 = max(j1-(ceil(rmin)-1),1):min(j1+(ceil(rmin)-1),nely)
                    e2 = (i2-1)*nely+j2;
                    k = k+1;
                    iH(k) = e1;
                    jH(k) = e2;
                    sH(k) = max(0, rmin-sqrt((i1-i2)^2+(j1-j2)^2));
                end
            end
        end
    end
    H = sparse(iH, jH, sH);
    Hs = sum(H, 2);

    F = sparse((nely+1)*(nelx+1), 1);
    for i = 1:nely+1
        F(i, 1) = 1.0;
    end

    fixeddofs = [];
    for i = 1:nelx+1
        fixeddofs = [fixeddofs, i*(nely+1)];
    end
    alldofs = 1:(nely+1)*(nelx+1);
    freedofs = setdiff(alldofs, fixeddofs);

    T = zeros((nely+1)*(nelx+1), 1);

    c_history = zeros(maxloop, 1);
    v_history = zeros(maxloop, 1);

    while change > tolx && loop < maxloop
        loop = loop + 1;

        sK = reshape(KE(:)*(xPhys(:)'.^penal), 16*nelx*nely, 1);
        K = sparse(iK, jK, sK);
        K = (K + K')/2;

        T(freedofs) = K(freedofs, freedofs) \ F(freedofs);

        ce = reshape(sum((T(edofMat)*KE).*T(edofMat), 2), nely, nelx);
        c = sum(sum((xPhys.^penal).*ce));
        dc = -penal*(xPhys.^(penal-1)).*ce;
        dv = ones(nely, nelx);

        if ft == 1
            dc(:) = H*(x(:).*dc(:))./Hs./max(1e-3, x(:));
        elseif ft == 2
```

```
            dc(:) = H*(dc(:)./Hs);
            dv(:) = H*(dv(:)./Hs);
        end

        l1 = 0; l2 = 1e9; move = 0.2;
        while (l2-l1)/(l1+l2) > 1e-3
            lmid = 0.5*(l2+l1);
            xnew = max(0, max(x-move, min(1, min(x+move, x.*sqrt(-dc./dv/lmid)))));
            if ft == 1
                xPhys = xnew;
            elseif ft == 2
                xPhys(:) = (H*xnew(:))./Hs;
            end
            if sum(xPhys(:)) > volfrac*nelx*nely
                l1 = lmid;
            else
                l2 = lmid;
            end
        end
        change = max(abs(xnew(:)-x(:)));
        x = xnew;

        c_history(loop) = c;
        v_history(loop) = mean(xPhys(:));

        fprintf('Iter:%4i Obj:%10.4f Vol:%6.3f Ch:%6.3f\n', loop, c, mean(xPhys(:)),
    change);
    end

    figure('Position', [100, 100, 1200, 400]);

    subplot(1,3,1);
    colormap(gray);
    imagesc(1-xPhys);
    axis equal; axis tight; axis off;
    title('Optimized Topology');

    subplot(1,3,2);
    plot(1:loop, c_history(1:loop), 'b-', 'LineWidth', 1.5);
    xlabel('Iteration');
    ylabel('Objective Function');
    title('Convergence History');
    grid on;

    subplot(1,3,3);
    plot(1:loop, v_history(1:loop), 'r-', 'LineWidth', 1.5);
    xlabel('Iteration');
    ylabel('Volume Fraction');
    title('Volume History');
    grid on;

    xPhys_filtered = image_reconstruction(xPhys, 0.5, 50);

    figure;
    colormap(gray);
    imagesc(1-xPhys_filtered);
    axis equal; axis tight; axis off;
    title('Reconstructed Topology');

    save('topology_result.mat', 'xPhys', 'xPhys_filtered', 'c_history', 'v_history');
    export_to_cad(xPhys_filtered, 'fin_geometry.dxf');
end

function [KE] = lk_heat()
```

```
    k = [2/3 -1/6 -1/3 -1/6
        -1/6  2/3 -1/6 -1/3
        -1/3 -1/6  2/3 -1/6
        -1/6 -1/3 -1/6  2/3];
    KE = k;
end

function xPhys_filtered = image_reconstruction(xPhys, mu, beta)
    [nely, nelx] = size(xPhys);
    xPhys_filtered = zeros(nely, nelx);

    for i = 1:nely
        for j = 1:nelx
            xf = xPhys(i, j);
            xp = (tanh(beta*mu) + tanh(beta*(xf - mu))) / (tanh(beta*mu) + tanh(beta*(1 -
    mu)));
            xPhys_filtered(i, j) = xp;
        end
    end

    threshold = 0.5;
    xPhys_filtered(xPhys_filtered >= threshold) = 1;
    xPhys_filtered(xPhys_filtered < threshold) = 0;

    se = strel('disk', 2);
    xPhys_filtered = imclose(xPhys_filtered, se);
    xPhys_filtered = imopen(xPhys_filtered, se);

    xPhys_filtered = medfilt2(xPhys_filtered, [3 3]);
end

function export_to_cad(xPhys, filename)
    [nely, nelx] = size(xPhys);
    boundaries = bwboundaries(xPhys);

    fid = fopen(filename, 'w');
    fprintf(fid, '0\nSECTION\n2\nENTITIES\n');

    for k = 1:length(boundaries)
        boundary = boundaries{k};
        n_points = size(boundary, 1);

        for i = 1:n_points-1
            fprintf(fid, '0\nLINE\n8\n0\n');
            fprintf(fid, '10\n%f\n20\n%f\n30\n0.0\n', boundary(i,2), nely-boundary(i,1));
            fprintf(fid, '11\n%f\n21\n%f\n31\n0.0\n', boundary(i+1,2), nely-boundary(i
   +1,1));
        end
    end

    fprintf(fid, '0\nENDSEC\n0\nEOF\n');
    fclose(fid);
end

function MMA_optimization()
    nelx = 100;
    nely = 100;
    volfrac = 0.05;
    penal = 3;
    rmin = 2.5;

    m = 1;
    n = nelx * nely;
    xmin = zeros(n, 1);
```

```
    xmax = ones(n, 1);
    xold1 = volfrac * ones(n, 1);
    xold2 = volfrac * ones(n, 1);
    x = volfrac * ones(n, 1);

    low = zeros(n, 1);
    upp = ones(n, 1);

    a0 = 1;
    a = zeros(m, 1);
    c_mma = 1000 * ones(m, 1);
    d = zeros(m, 1);

    loop = 0;
    change = 1;
    maxloop = 500;

    [KE] = lk_heat();
    nodenrs = reshape(1:(1+nelx)*(1+nely), 1+nely, 1+nelx);
    edofVec = reshape(nodenrs(1:end-1, 1:end-1)+1, nelx*nely, 1);
    edofMat = repmat(edofVec, 1, 4) + repmat([0 nely+[1 0] -1], nelx*nely, 1);
    iK = reshape(kron(edofMat, ones(4,1))', 16*nelx*nely, 1);
    jK = reshape(kron(edofMat, ones(1,4))', 16*nelx*nely, 1);

    F = sparse((nely+1)*(nelx+1), 1);
    F(1:nely+1, 1) = 1;
    fixeddofs = (nely+1)*(nelx+1);
    alldofs = 1:(nely+1)*(nelx+1);
    freedofs = setdiff(alldofs, fixeddofs);
    T = zeros((nely+1)*(nelx+1), 1);

    while change > 1e-3 && loop < maxloop
        loop = loop + 1;

        xPhys = reshape(x, nely, nelx);
        sK = reshape(KE(:)*(xPhys(:)'.^penal), 16*nelx*nely, 1);
        K = sparse(iK, jK, sK);
        K = (K + K')/2;
        T(freedofs) = K(freedofs, freedofs) \ F(freedofs);

        ce = reshape(sum((T(edofMat)*KE).*T(edofMat), 2), nely, nelx);
        f0val = sum(sum((xPhys.^penal).*ce));
        df0dx = -penal*(xPhys.^(penal-1)).*ce;
        df0dx = df0dx(:);

        fval = sum(x) / (volfrac * n) - 1;
        dfdx = ones(n, 1) / (volfrac * n);

        [xmma, ~, ~, ~, ~, ~, ~, ~, ~, low, upp] = mmasub(m, n, loop, x, xmin, xmax, ...
            xold1, xold2, f0val, df0dx, fval, dfdx', low, upp, a0, a, c_mma, d);

        xold2 = xold1;
        xold1 = x;
        change = max(abs(xmma - x));
        x = xmma;

        fprintf('MMA Iter:%4i Obj:%10.4f Vol:%6.3f Ch:%6.3f\n', loop, f0val, mean(x),
    change);
    end
end

function [xmma,ymma,zmma,lam,xsi,eta,mu,zet,s,low,upp] = mmasub(m,n,iter,xval,xmin,xmax,
    xold1,xold2,f0val,df0dx,fval,dfdx,low,upp,a0,a,c,d)
    epsimin = 1e-7;
```

```
    raa0 = 0.00001;
    albefa = 0.1;
    asyinit = 0.5;
    asyincr = 1.2;
    asydecr = 0.7;

    eeen = ones(n,1);
    eeem = ones(m,1);
    zeron = zeros(n,1);

    if iter < 2.5
        low = xval - asyinit*(xmax-xmin);
        upp = xval + asyinit*(xmax-xmin);
    else
        zzz = (xval-xold1).*(xold1-xold2);
        factor = eeen;
        factor(zzz > 0) = asyincr;
        factor(zzz < 0) = asydecr;
        low = xval - factor.*(xold1 - low);
        upp = xval + factor.*(upp - xold1);
        lowmin = xval - 10*(xmax-xmin);
        lowmax = xval - 0.01*(xmax-xmin);
        uppmin = xval + 0.01*(xmax-xmin);
        uppmax = xval + 10*(xmax-xmin);
        low = max(low, lowmin);
        low = min(low, lowmax);
        upp = min(upp, uppmax);
        upp = max(upp, uppmin);
    end

    zzz1 = low + albefa*(xval-low);
    zzz2 = xval - (xmax-xmin);
    zzz = max(zzz1, zzz2);
    alfa = max(zzz, xmin);
    zzz1 = upp - albefa*(upp-xval);
    zzz2 = xval + (xmax-xmin);
    zzz = min(zzz1, zzz2);
    beta = min(zzz, xmax);

    xmma = 0.5*(alfa+beta);
    ymma = eeem;
    zmma = 1;
    lam = eeem;
    xsi = eeen./(xmma-alfa);
    xsi = max(xsi, eeen);
    eta = eeen./(beta-xmma);
    eta = max(eta, eeen);
    mu = max(eeem, 0.5*c);
    zet = 1;
    s = eeem;
end

function analyze_results(xPhys)
    [nely, nelx] = size(xPhys);

    total_area = nelx * nely;
    fin_area = sum(xPhys(:));
    volume_fraction = fin_area / total_area;

    boundaries = bwboundaries(xPhys);
    total_perimeter = 0;
    for k = 1:length(boundaries)
        boundary = boundaries{k};
        for i = 1:size(boundary,1)-1
```

```
            total_perimeter = total_perimeter + norm(boundary(i+1,:) - boundary(i,:));
        end
    end

    props = regionprops(xPhys, 'Centroid', 'Area', 'Perimeter', 'Eccentricity');

    fprintf('Volume Fraction: %.4f\n', volume_fraction);
    fprintf('Total Perimeter: %.2f\n', total_perimeter);
    fprintf('Number of Components: %d\n', length(props));

    for i = 1:length(props)
        fprintf('Component %d: Area=%.2f, Perimeter=%.2f, Eccentricity=%.4f\n', ...
            i, props(i).Area, props(i).Perimeter, props(i).Eccentricity);
    end
end

function thermal_analysis(xPhys, k_fin, k_pcm, T_hot, T_init)
    [nely, nelx] = size(xPhys);

    k_eff = xPhys * k_fin + (1 - xPhys) * k_pcm;

    ndof = (nelx+1) * (nely+1);
    K_global = sparse(ndof, ndof);
    F_global = sparse(ndof, 1);

    [KE_fin] = lk_heat() * k_fin;
    [KE_pcm] = lk_heat() * k_pcm;

    nodenrs = reshape(1:(1+nelx)*(1+nely), 1+nely, 1+nelx);
    edofVec = reshape(nodenrs(1:end-1, 1:end-1)+1, nelx*nely, 1);
    edofMat = repmat(edofVec, 1, 4) + repmat([0 nely+[1 0] -1], nelx*nely, 1);

    for el = 1:nelx*nely
        [row, col] = ind2sub([nely, nelx], el);
        edof = edofMat(el, :);

        if xPhys(row, col) > 0.5
            KE = KE_fin;
        else
            KE = KE_pcm;
        end

        K_global(edof, edof) = K_global(edof, edof) + KE;
    end

    fixed_nodes = 1:nely+1;
    free_nodes = setdiff(1:ndof, fixed_nodes);

    T = zeros(ndof, 1);
    T(fixed_nodes) = T_hot;

    F_global = F_global - K_global(:, fixed_nodes) * T(fixed_nodes);
    T(free_nodes) = K_global(free_nodes, free_nodes) \ F_global(free_nodes);

    T_field = reshape(T, nely+1, nelx+1);

    figure;
    contourf(T_field, 50);
    colorbar;
    title('Temperature Distribution');
    xlabel('x');
    ylabel('y');

    q_total = sum(K_global(fixed_nodes, :) * T);
```

```
    fprintf('Total Heat Flux: %.4f W\n', q_total);
    fprintf('Average Temperature: %.2f K\n', mean(T));
    fprintf('Max Temperature: %.2f K\n', max(T));
    fprintf('Min Temperature: %.2f K\n', min(T));
end
```

Appendix B: MATLAB Code for Fractal Fin Generation

```
function FractalFinGeneration()
    R1 = 0.02;
    R2 = 0.05;
    n_branches = 15;
    levels = 3;
    L0 = 0.018;
    W0 = 0.006;
    alpha = 0.6;
    gamma = 2;

    [fins, nodes] = generate_fractal_fins(n_branches, levels, L0, W0, alpha, gamma, R1,
    R2);

    figure('Position', [100, 100, 800, 800]);
    hold on;
    axis equal;

    theta_circle = linspace(0, 2*pi, 100);
    plot(R1*cos(theta_circle), R1*sin(theta_circle), 'b-', 'LineWidth', 2);
    plot(R2*cos(theta_circle), R2*sin(theta_circle), 'b-', 'LineWidth', 2);

    for i = 1:size(fins, 1)
        x1 = fins(i, 1);
        y1 = fins(i, 2);
        x2 = fins(i, 3);
        y2 = fins(i, 4);
        w = fins(i, 5);

        draw_rectangle_fin(x1, y1, x2, y2, w);
    end

    title('Fractal Fin Distribution');
    xlabel('x (m)');
    ylabel('y (m)');
    hold off;

    export_fractal_to_ansys(fins, 'fractal_fins.txt');
    export_fractal_to_dxf(fins, 'fractal_fins.dxf');

    [total_area, total_length, surface_area] = analyze_fractal_fins(fins);
    fprintf('Total Fin Area: %.6f m^2\n', total_area);
    fprintf('Total Fin Length: %.4f m\n', total_length);
    fprintf('Total Surface Area: %.6f m^2\n', surface_area);
end

function [fins, nodes] = generate_fractal_fins(n_branches, levels, L0, W0, alpha, gamma,
    R1, R2)
    fins = [];
    nodes = [];
    theta_base = 2*pi / n_branches;

    for i = 1:n_branches
```

```
        theta = (i-1) * theta_base;
        start_point = [R1*cos(theta), R1*sin(theta)];
        direction = [cos(theta), sin(theta)];

        [branch_fins, branch_nodes] = generate_branch(start_point, direction, L0, W0, alpha, gamma, levels, R2, 0);
        fins = [fins; branch_fins];
        nodes = [nodes; branch_nodes];
    end
end

function [branch_fins, branch_nodes] = generate_branch(start, dir, L, W, alpha, gamma, level, R_max, current_level)
    branch_fins = [];
    branch_nodes = [];

    if level == 0
        return;
    end

    end_point = start + L * dir;

    if norm(end_point) > R_max
        scale = (R_max - norm(start)) / (norm(end_point) - norm(start));
        end_point = start + scale * L * dir;
        L = scale * L;
    end

    branch_fins = [branch_fins; start(1), start(2), end_point(1), end_point(2), W];
    branch_nodes = [branch_nodes; start(1), start(2), current_level; end_point(1), end_point(2), current_level];

    if norm(end_point) >= R_max * 0.95
        return;
    end

    W_child = W * (1/3)^(1/gamma);
    L_child = alpha * L;

    n_children = 2;
    angle_spread = pi/6;

    for j = 1:n_children
        if n_children == 2
            angle_offset = (j - 1.5) * 2 * angle_spread;
        else
            angle_offset = (j - (n_children+1)/2) * angle_spread;
        end

        dir_child = rotate_vector(dir, angle_offset);

        [child_fins, child_nodes] = generate_branch(end_point, dir_child, L_child, W_child, alpha, gamma, level-1, R_max, current_level+1);
        branch_fins = [branch_fins; child_fins];
        branch_nodes = [branch_nodes; child_nodes];
    end
end

function [v_rot] = rotate_vector(v, theta)
    R = [cos(theta), -sin(theta); sin(theta), cos(theta)];
    v_rot = (R * v')';
end

function draw_rectangle_fin(x1, y1, x2, y2, w)
```

```
    L = sqrt((x2-x1)^2 + (y2-y1)^2);

    if L < 1e-10
        return;
    end

    dx = (x2 - x1) / L;
    dy = (y2 - y1) / L;

    nx = -dy;
    ny = dx;

    corners_x = [x1 + w/2*nx, x2 + w/2*nx, x2 - w/2*nx, x1 - w/2*nx, x1 + w/2*nx];
    corners_y = [y1 + w/2*ny, y2 + w/2*ny, y2 - w/2*ny, y1 - w/2*ny, y1 + w/2*ny];

    fill(corners_x, corners_y, [0.7 0.7 0.7], 'EdgeColor', 'k', 'LineWidth', 0.5);
end

function export_fractal_to_ansys(fins, filename)
    fid = fopen(filename, 'w');

    fprintf(fid, 'FRACTAL FIN GEOMETRY DATA\n');
    fprintf(fid, 'Number of fin segments: %d\n', size(fins, 1));
    fprintf(fid, '================================\n');
    fprintf(fid, 'X1\tY1\tX2\tY2\tWidth\n');

    for i = 1:size(fins, 1)
        fprintf(fid, '%.8f\t%.8f\t%.8f\t%.8f\t%.8f\n', ...
            fins(i,1), fins(i,2), fins(i,3), fins(i,4), fins(i,5));
    end

    fclose(fid);
end

function export_fractal_to_dxf(fins, filename)
    fid = fopen(filename, 'w');

    fprintf(fid, '0\nSECTION\n2\nHEADER\n');
    fprintf(fid, '9\n$ACADVER\n1\nAC1015\n');
    fprintf(fid, '0\nENDSEC\n');
    fprintf(fid, '0\nSECTION\n2\nENTITIES\n');

    for i = 1:size(fins, 1)
        x1 = fins(i, 1);
        y1 = fins(i, 2);
        x2 = fins(i, 3);
        y2 = fins(i, 4);
        w = fins(i, 5);

        L = sqrt((x2-x1)^2 + (y2-y1)^2);
        if L < 1e-10
            continue;
        end

        dx = (x2 - x1) / L;
        dy = (y2 - y1) / L;
        nx = -dy;
        ny = dx;

        corners_x = [x1 + w/2*nx, x2 + w/2*nx, x2 - w/2*nx, x1 - w/2*nx];
        corners_y = [y1 + w/2*ny, y2 + w/2*ny, y2 - w/2*ny, y1 - w/2*ny];

        fprintf(fid, '0\nLWPOLYLINE\n8\nFINS\n70\n1\n90\n4\n');
        for j = 1:4
```

```
            fprintf(fid, '10\n%.8f\n20\n%.8f\n', corners_x(j), corners_y(j));
        end
    end

    fprintf(fid, '0\nENDSEC\n0\nEOF\n');
    fclose(fid);
end

function [total_area, total_length, surface_area] = analyze_fractal_fins(fins)
    total_area = 0;
    total_length = 0;
    surface_area = 0;

    for i = 1:size(fins, 1)
        x1 = fins(i, 1);
        y1 = fins(i, 2);
        x2 = fins(i, 3);
        y2 = fins(i, 4);
        w = fins(i, 5);

        L = sqrt((x2-x1)^2 + (y2-y1)^2);

        total_length = total_length + L;
        total_area = total_area + L * w;
        surface_area = surface_area + 2 * (L + w);
    end
end

function optimize_fractal_parameters()
    R1 = 0.02;
    R2 = 0.05;
    n_branches_range = 10:2:20;
    levels_range = 2:4;
    alpha_range = 0.4:0.1:0.8;
    gamma_range = 2:4;

    best_performance = 0;
    best_params = struct();

    L0 = 0.018;
    W0 = 0.006;

    results = [];

    for n_branches = n_branches_range
        for levels = levels_range
            for alpha = alpha_range
                for gamma = gamma_range
                    [fins, ~] = generate_fractal_fins(n_branches, levels, L0, W0, alpha,
    gamma, R1, R2);

                    [total_area, total_length, surface_area] = analyze_fractal_fins(fins)
    ;

                    tank_area = pi * (R2^2 - R1^2);
                    volume_fraction = total_area / tank_area;

                    if volume_fraction > 0.15
                        continue;
                    end

                    performance = surface_area / total_area;

                    results = [results; n_branches, levels, alpha, gamma, ...
```

```
                        total_area, total_length, surface_area, volume_fraction,
    performance];

                    if performance > best_performance
                        best_performance = performance;
                        best_params.n_branches = n_branches;
                        best_params.levels = levels;
                        best_params.alpha = alpha;
                        best_params.gamma = gamma;
                    end
                end
            end
        end
    end

    fprintf('Best Parameters:\n');
    fprintf('  n_branches: %d\n', best_params.n_branches);
    fprintf('  levels: %d\n', best_params.levels);
    fprintf('  alpha: %.2f\n', best_params.alpha);
    fprintf('  gamma: %d\n', best_params.gamma);
    fprintf('  Performance Index: %.4f\n', best_performance);
end

function generate_murray_optimal_fins()
    R1 = 0.02;
    R2 = 0.05;
    n_main = 15;

    Q_total = 1.0;
    Q_branch = Q_total / n_main;

    gamma = 2;

    W0 = (Q_total)^(1/gamma) * 0.001;
    W1 = (Q_branch)^(1/gamma) * 0.001;

    L0 = (R2 - R1) * 0.6;

    alpha = 0.6;
    beta_angle = pi/6;

    fins = [];

    for i = 1:n_main
        theta = (i-1) * 2 * pi / n_main;

        start_x = R1 * cos(theta);
        start_y = R1 * sin(theta);

        end_x = start_x + L0 * cos(theta);
        end_y = start_y + L0 * sin(theta);

        fins = [fins; start_x, start_y, end_x, end_y, W0];

        L1 = alpha * L0;
        W1_child = W0 * (1/2)^(1/gamma);

        for j = [-1, 1]
            child_theta = theta + j * beta_angle;
            child_end_x = end_x + L1 * cos(child_theta);
            child_end_y = end_y + L1 * sin(child_theta);

            if sqrt(child_end_x^2 + child_end_y^2) <= R2
                fins = [fins; end_x, end_y, child_end_x, child_end_y, W1_child];
```

```
                L2 = alpha * L1;
                W2_child = W1_child * (1/2)^(1/gamma);

                for k = [-1, 1]
                    grandchild_theta = child_theta + k * beta_angle * 0.8;
                    grandchild_end_x = child_end_x + L2 * cos(grandchild_theta);
                    grandchild_end_y = child_end_y + L2 * sin(grandchild_theta);

                    if sqrt(grandchild_end_x^2 + grandchild_end_y^2) <= R2
                        fins = [fins; child_end_x, child_end_y, grandchild_end_x,
    grandchild_end_y, W2_child];
                    end
                end
            end
        end
    end

    visualize_fins(fins, R1, R2);
end

function visualize_fins(fins, R1, R2)
    figure('Position', [100, 100, 800, 800]);
    hold on;
    axis equal;

    theta_circle = linspace(0, 2*pi, 200);
    fill(R2*cos(theta_circle), R2*sin(theta_circle), [0.9 0.95 1], 'EdgeColor', 'b', '
    LineWidth', 2);
    fill(R1*cos(theta_circle), R1*sin(theta_circle), [1 0.8 0.8], 'EdgeColor', 'r', '
    LineWidth', 2);

    for i = 1:size(fins, 1)
        draw_rectangle_fin(fins(i,1), fins(i,2), fins(i,3), fins(i,4), fins(i,5));
    end

    title('Murray Law Optimized Fractal Fins', 'FontSize', 14);
    xlabel('x (m)', 'FontSize', 12);
    ylabel('y (m)', 'FontSize', 12);

    xlim([-R2*1.1, R2*1.1]);
    ylim([-R2*1.1, R2*1.1]);

    grid on;
    hold off;
end
```

Appendix C: ANSYS Fluent UDF for Phase Change Simulation

```
#include "udf.h"
#include "mem.h"
#include "metric.h"

#define Tmelt1 273.15
#define Tmelt2 333.0
#define Latent 200000.0
#define Amush 1e7
#define epsilon 1e-3
#define beta_expansion 0.0002
#define rho_solid 850.0
```

```
#define rho_liquid 780.0
#define cp_solid 1800.0
#define cp_liquid 2000.0
#define k_solid 0.24
#define k_liquid 0.15
#define mu_liquid 0.023

static real liquid_fraction_field[10000];
static real temperature_field[10000];
static int cell_count = 0;

DEFINE_INIT(initialize_pcm, domain)
{
    Thread *t;
    cell_t c;
    real T_init = 293.15;

    thread_loop_c(t, domain)
    {
        begin_c_loop(c, t)
        {
            C_T(c, t) = T_init;
            C_UDMI(c, t, 0) = 0.0;
            C_UDMI(c, t, 1) = 0.0;
            C_UDMI(c, t, 2) = T_init;
        }
        end_c_loop(c, t)
    }
}

real calculate_liquid_fraction(real T)
{
    real fl;

    if (T <= Tmelt1)
    {
        fl = 0.0;
    }
    else if (T >= Tmelt2)
    {
        fl = 1.0;
    }
    else
    {
        fl = (T - Tmelt1) / (Tmelt2 - Tmelt1);
    }

    return fl;
}

real calculate_enthalpy(real T, real fl)
{
    real h_sensible, h_latent, h_total;

    if (T <= Tmelt1)
    {
        h_sensible = cp_solid * (T - 273.15);
        h_latent = 0.0;
    }
    else if (T >= Tmelt2)
    {
        h_sensible = cp_solid * (Tmelt1 - 273.15) + cp_liquid * (T - Tmelt2);
        h_latent = Latent;
    }
```

```
    else
    {
        h_sensible = cp_solid * (Tmelt1 - 273.15) + cp_liquid * (T - Tmelt1);
        h_latent = fl * Latent;
    }

    h_total = h_sensible + h_latent;

    return h_total;
}

DEFINE_SOURCE(mom_source_x, cell, thread, dS, eqn)
{
    real source;
    real T = C_T(cell, thread);
    real fl = calculate_liquid_fraction(T);
    real u = C_U(cell, thread);
    real damping;

    damping = Amush * pow(1.0 - fl, 2) / (pow(fl, 3) + epsilon);
    source = -damping * u;
    dS[eqn] = -damping;

    C_UDMI(cell, thread, 0) = fl;

    return source;
}

DEFINE_SOURCE(mom_source_y, cell, thread, dS, eqn)
{
    real source;
    real T = C_T(cell, thread);
    real fl = calculate_liquid_fraction(T);
    real v = C_V(cell, thread);
    real rho = C_R(cell, thread);
    real damping;
    real buoyancy;
    real T_ref = (Tmelt1 + Tmelt2) / 2.0;

    damping = Amush * pow(1.0 - fl, 2) / (pow(fl, 3) + epsilon);
    buoyancy = rho * 9.81 * beta_expansion * (T - T_ref);

    source = -damping * v + buoyancy;
    dS[eqn] = -damping;

    return source;
}

DEFINE_SOURCE(energy_source, cell, thread, dS, eqn)
{
    real source;
    real T = C_T(cell, thread);
    real T_old = C_T_M1(cell, thread);
    real rho = C_R(cell, thread);
    real dt = CURRENT_TIMESTEP;
    real fl, fl_old;
    real dfl_dt;

    fl = calculate_liquid_fraction(T);
    fl_old = calculate_liquid_fraction(T_old);

    if (dt > 0.0)
    {
        dfl_dt = (fl - fl_old) / dt;
```

```
    }
    else
    {
        dfl_dt = 0.0;
    }

    source = -rho * Latent * dfl_dt;

    if (T > Tmelt1 && T < Tmelt2)
    {
        dS[eqn] = -rho * Latent / (dt * (Tmelt2 - Tmelt1));
    }
    else
    {
        dS[eqn] = 0.0;
    }

    C_UDMI(cell, thread, 1) = calculate_enthalpy(T, fl);

    return source;
}

DEFINE_PROPERTY(pcm_density, cell, thread)
{
    real T = C_T(cell, thread);
    real fl = calculate_liquid_fraction(T);
    real rho;

    rho = rho_solid * (1.0 - fl) + rho_liquid * fl;

    return rho;
}

DEFINE_PROPERTY(pcm_specific_heat, cell, thread)
{
    real T = C_T(cell, thread);
    real fl = calculate_liquid_fraction(T);
    real cp;

    cp = cp_solid * (1.0 - fl) + cp_liquid * fl;

    if (T > Tmelt1 && T < Tmelt2)
    {
        cp = cp + Latent / (Tmelt2 - Tmelt1);
    }

    return cp;
}

DEFINE_PROPERTY(pcm_thermal_conductivity, cell, thread)
{
    real T = C_T(cell, thread);
    real fl = calculate_liquid_fraction(T);
    real k;

    k = k_solid * (1.0 - fl) + k_liquid * fl;

    return k;
}

DEFINE_PROPERTY(pcm_viscosity, cell, thread)
{
    real T = C_T(cell, thread);
    real fl = calculate_liquid_fraction(T);
```

```
    real mu;
    real mu_large = 1e5;

    if (fl < epsilon)
    {
        mu = mu_large;
    }
    else
    {
        mu = mu_liquid / fl;
        if (mu > mu_large)
        {
            mu = mu_large;
        }
    }

    return mu;
}

DEFINE_ADJUST(adjust_phase_change, domain)
{
    Thread *t;
    cell_t c;
    real fl_total = 0.0;
    real vol_total = 0.0;
    real T_avg = 0.0;
    real fl_avg;
    int count = 0;

    thread_loop_c(t, domain)
    {
        begin_c_loop(c, t)
        {
            real T = C_T(c, t);
            real fl = calculate_liquid_fraction(T);
            real vol = C_VOLUME(c, t);

            fl_total += fl * vol;
            vol_total += vol;
            T_avg += T * vol;
            count++;

            C_UDMI(c, t, 2) = T;
        }
        end_c_loop(c, t)
    }

    if (vol_total > 0.0)
    {
        fl_avg = fl_total / vol_total;
        T_avg = T_avg / vol_total;
    }
    else
    {
        fl_avg = 0.0;
        T_avg = 293.15;
    }

    Message("Time: %.2f s, Liquid Fraction: %.4f, Avg Temp: %.2f K\n",
            CURRENT_TIME, fl_avg, T_avg);
}

DEFINE_EXECUTE_AT_END(output_results)
{
```

```
    Domain *domain = Get_Domain(1);
    Thread *t;
    cell_t c;
    real fl_total = 0.0;
    real vol_total = 0.0;
    real T_max = 0.0;
    real T_min = 1e10;
    real T_avg = 0.0;
    real energy_stored = 0.0;
    FILE *fp;

    thread_loop_c(t, domain)
    {
        begin_c_loop(c, t)
        {
            real T = C_T(c, t);
            real fl = calculate_liquid_fraction(T);
            real vol = C_VOLUME(c, t);
            real rho = C_R(c, t);
            real h = calculate_enthalpy(T, fl);

            fl_total += fl * vol;
            vol_total += vol;
            T_avg += T * vol;
            energy_stored += rho * h * vol;

            if (T > T_max) T_max = T;
            if (T < T_min) T_min = T;
        }
        end_c_loop(c, t)
    }

    if (vol_total > 0.0)
    {
        fl_total = fl_total / vol_total;
        T_avg = T_avg / vol_total;
    }

    fp = fopen("phase_change_results.txt", "a");
    if (fp != NULL)
    {
        fprintf(fp, "%.4f\t%.6f\t%.2f\t%.2f\t%.2f\t%.2f\n",
                CURRENT_TIME, fl_total, T_avg, T_min, T_max, energy_stored);
        fclose(fp);
    }
}

DEFINE_ON_DEMAND(calculate_melting_time)
{
    Domain *domain = Get_Domain(1);
    Thread *t;
    cell_t c;
    real fl_total = 0.0;
    real vol_total = 0.0;
    real target_fl = 0.99;

    thread_loop_c(t, domain)
    {
        begin_c_loop(c, t)
        {
            real T = C_T(c, t);
            real fl = calculate_liquid_fraction(T);
            real vol = C_VOLUME(c, t);

```

```
            fl_total += fl * vol;
            vol_total += vol;
        }
        end_c_loop(c, t)
    }

    if (vol_total > 0.0)
    {
        fl_total = fl_total / vol_total;
    }

    Message("Current Time: %.2f s\n", CURRENT_TIME);
    Message("Average Liquid Fraction: %.4f\n", fl_total);

    if (fl_total >= target_fl)
    {
        Message("PCM fully melted at time: %.2f s\n", CURRENT_TIME);
    }
}

DEFINE_PROFILE(hot_wall_temperature, thread, position)
{
    face_t f;
    real T_wall = 373.15;

    begin_f_loop(f, thread)
    {
        F_PROFILE(f, thread, position) = T_wall;
    }
    end_f_loop(f, thread)
}

DEFINE_PROFILE(time_varying_temperature, thread, position)
{
    face_t f;
    real time = CURRENT_TIME;
    real T_initial = 293.15;
    real T_final = 373.15;
    real ramp_time = 60.0;
    real T_wall;

    if (time < ramp_time)
    {
        T_wall = T_initial + (T_final - T_initial) * time / ramp_time;
    }
    else
    {
        T_wall = T_final;
    }

    begin_f_loop(f, thread)
    {
        F_PROFILE(f, thread, position) = T_wall;
    }
    end_f_loop(f, thread)
}

DEFINE_DIFFUSIVITY(pcm_diffusivity, cell, thread, eqn_index)
{
    real T = C_T(cell, thread);
    real fl = calculate_liquid_fraction(T);
    real k, rho, cp;
    real alpha_thermal;

```

```
    k = k_solid * (1.0 - fl) + k_liquid * fl;
    rho = rho_solid * (1.0 - fl) + rho_liquid * fl;
    cp = cp_solid * (1.0 - fl) + cp_liquid * fl;

    if (T > Tmelt1 && T < Tmelt2)
    {
        cp = cp + Latent / (Tmelt2 - Tmelt1);
    }

    alpha_thermal = k / (rho * cp);

    return alpha_thermal;
}

DEFINE_EXECUTE_ON_LOADING(init_udf, libname)
{
    Message("Phase Change UDF Loaded Successfully\n");
    Message("Solidus Temperature: %.2f K\n", Tmelt1);
    Message("Liquidus Temperature: %.2f K\n", Tmelt2);
    Message("Latent Heat: %.0f J/kg\n", Latent);
    Message("Mushy Zone Constant: %.0e\n", Amush);
}
```